ARGO CD GITOPS › MY LEARNING NOTES › SESSION 02

Screenshots: First GitOps Application

Raw screenshots captured while watching this session (UI app creation, CLI deployment, declarative Application YAML, sync loop, troubleshooting, deletion), kept alongside the written notes (02-first-gitops-application.md) for visual reference.

65 screenshots · captured 2026-09-12, 10:24 AM - 1:20 PM

FIRST GITOPS APPLICATION



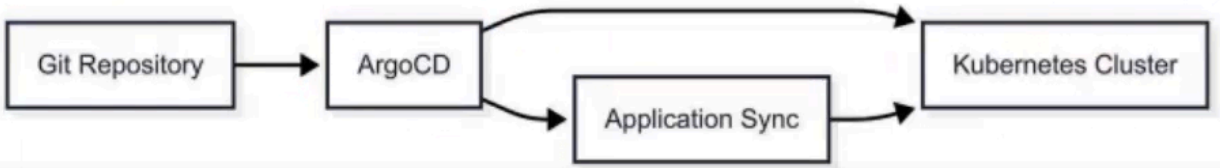
What are Argo CD Applications

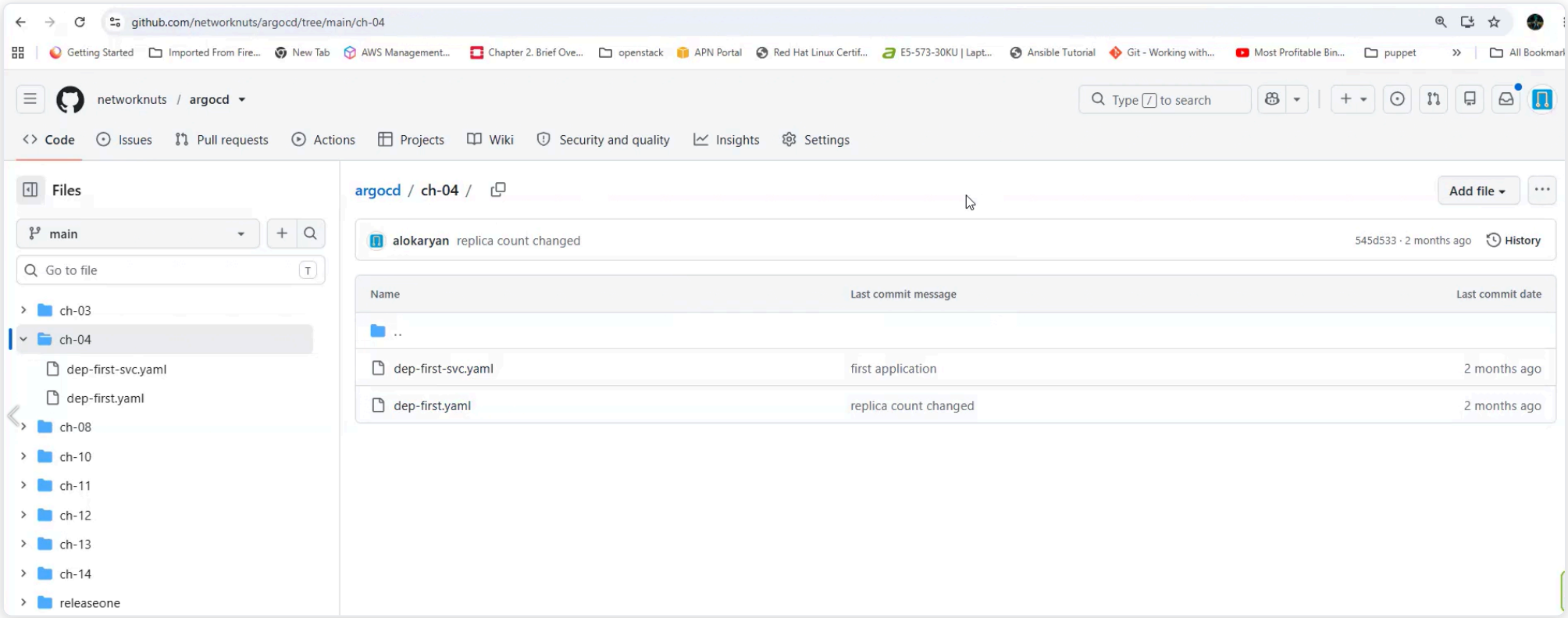
We will create our first application, connect it to a Git repository, and let Argo CD handle the deployment to your Kubernetes cluster. Start simple, by deploying a static NGINX site. The Kubernetes manifests for this site will live in a Git repository. Argo CD will watch the repo for changes and keep your cluster in sync.

Argo CD applications connect your Git repository to your Kubernetes cluster. They define what should run and where it should run.

An Argo CD application defines the link between your Git repository and your Kubernetes cluster. It describes:

- Source: The Git repo and path containing your manifests.
- Destination: The target cluster and namespace.
- Sync Policy: The rules for how Argo CD applies changes.




github.com/networknuts/argocd/tree/main/ch-04
networknuts / argocd
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file
ch-03
ch-04
dep-first-svc.yaml
dep-first.yaml
ch-08
ch-10
ch-11
ch-12
ch-13
ch-14
releaseone
argocd / ch-04 /
Add file
alokaryan replica count changed
545d533 · 2 months ago
History
Name
Last commit message
Last commit date
..
dep-first-svc.yaml
first application
2 months ago
dep-first.yaml
replica count changed
2 months ago

argocd / ch-04 / dep-first.yaml

alokaryan replica count changed

Code | Blame | 21 lines (21 loc) · 341 Bytes

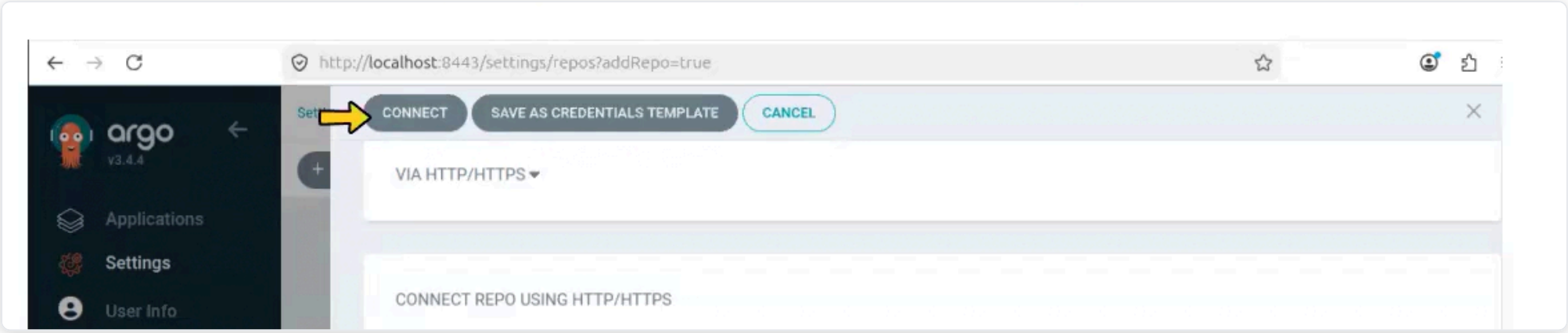
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 4
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

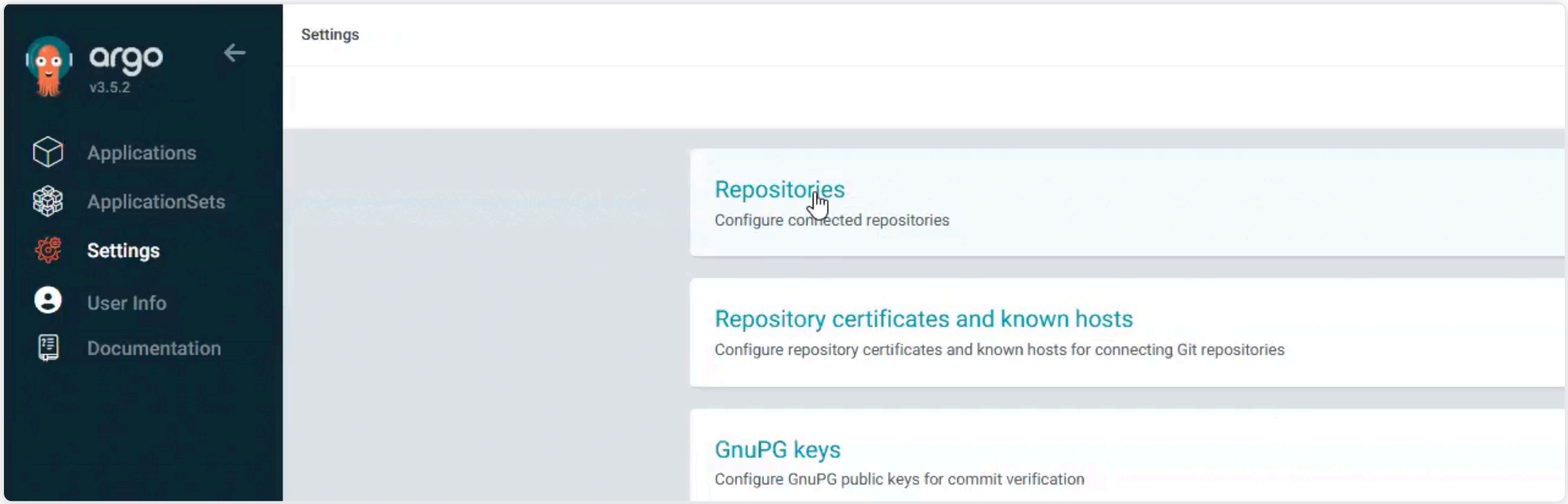
argocd / ch-04 / dep-first-svc.yaml

alokaryan first application

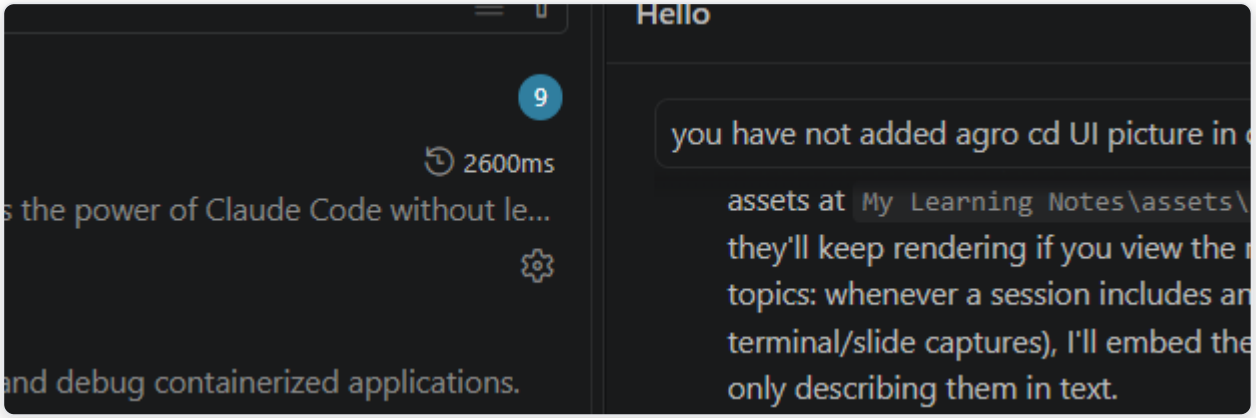
Code Blame 11 lines (11 loc) · 166 Bytes

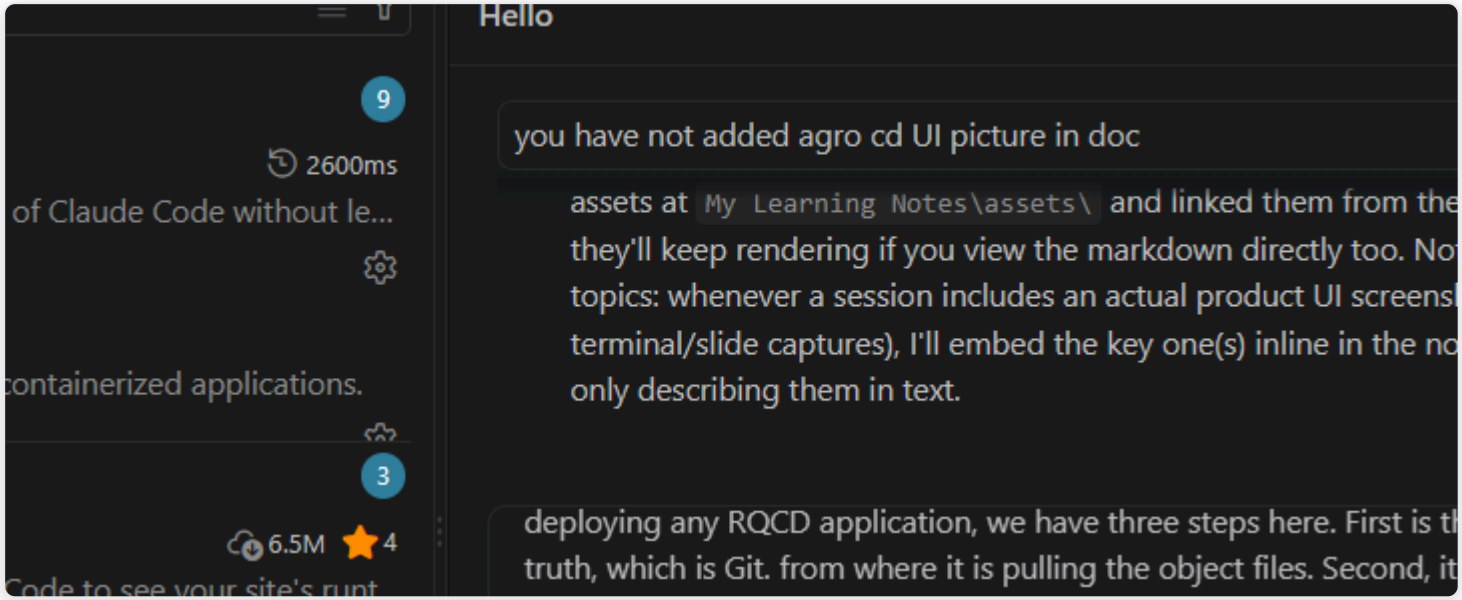
```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-deployment-svc
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

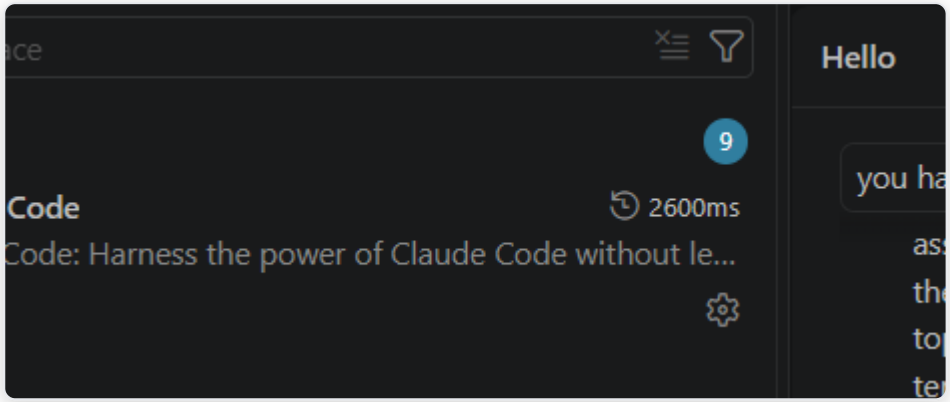

http://localhost:8443/settings/repos?addRepo=true
argo
v3.4.4
Applications
Settings
User Info
CONNECT
SAVE AS CREDENTIALS TEMPLATE
CANCEL
VIA HTTP/HTTPS
CONNECT REPO USING HTTP/HTTPS

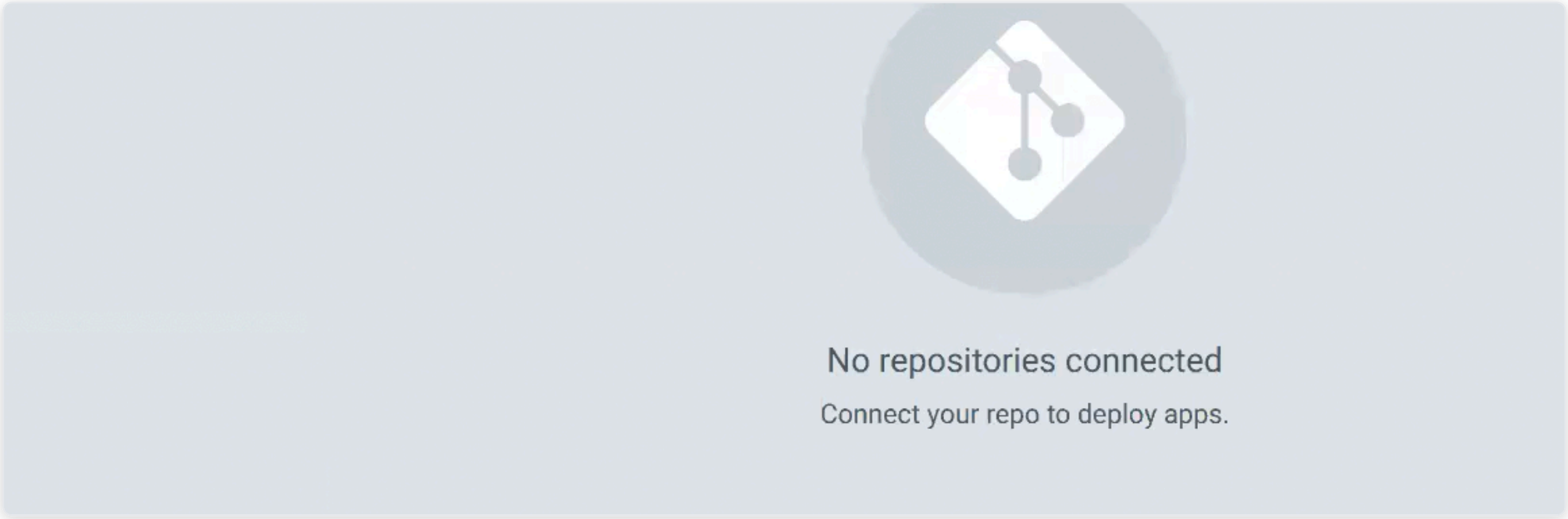

argo
v3.5.2
Applications
ApplicationSets
Settings
User Info
Documentation
Settings
Repositories
Configure connected repositories
Repository certificates and known hosts
Configure repository certificates and known hosts for connecting Git repositories
GnuPG keys
Configure GnuPG public keys for commit verification

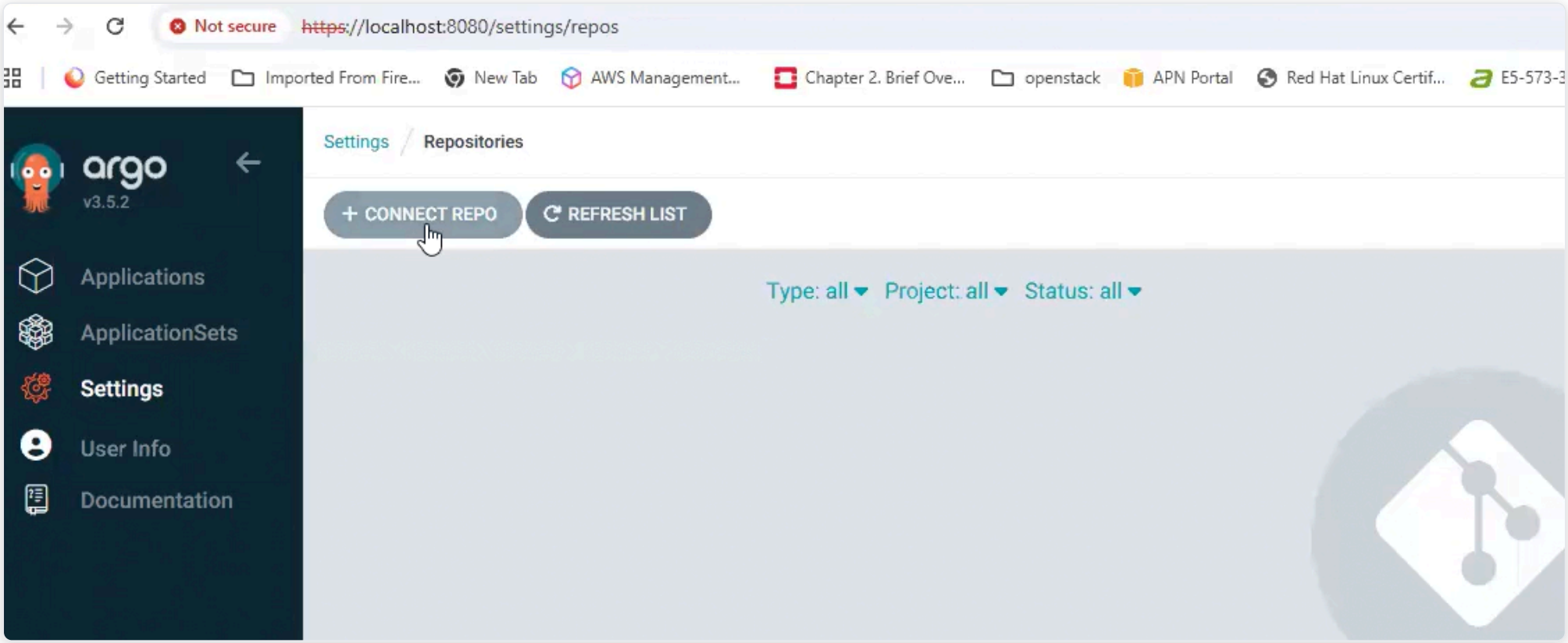


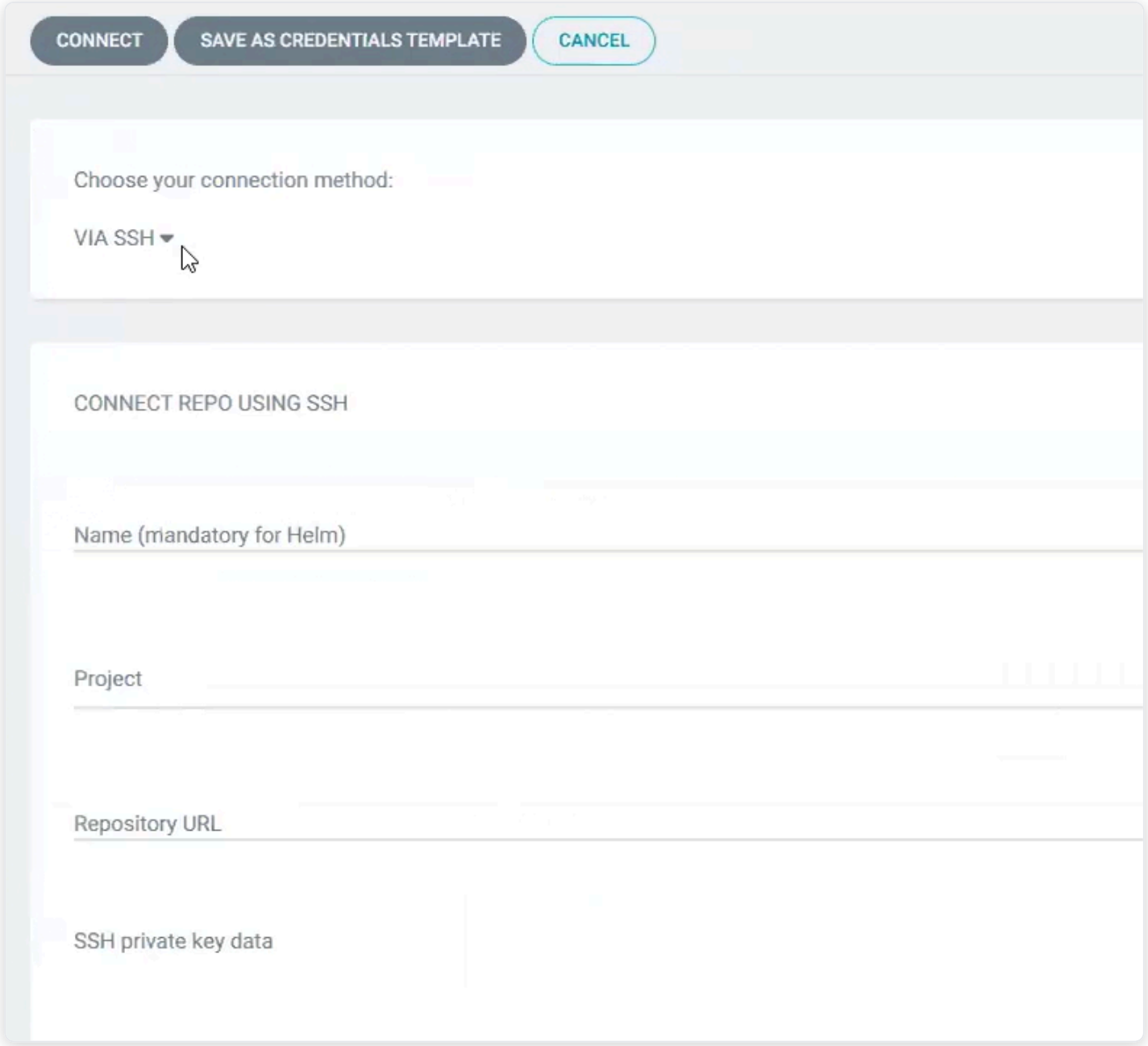

Hello
9
2600ms
s the power of Claude Code without le...
and debug containerized applications.
you have not added agro cd UI picture in
assets at My Learning Notes\assets\
they'll keep rendering if you view the
topics: whenever a session includes a
terminal/slide captures), I'll embed the
only describing them in text.

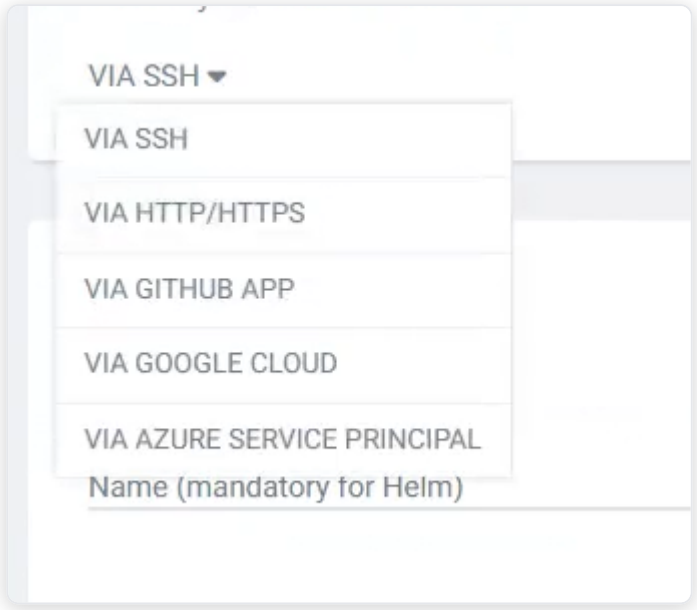

Hello
9
2600ms
of Claude Code without le...
containerized applications.
3
6.5M
4
Code to see your site's runt
you have not added agro cd UI picture in doc
assets at My Learning Notes\assets\ and linked them from the
they'll keep rendering if you view the markdown directly too. No
topics: whenever a session includes an actual product UI screens
terminal/slide captures), I'll embed the key one(s) inline in the no
only describing them in text.
deploying any RQCD application, we have three steps here. First is t
truth, which is Git. from where it is pulling the object files. Second, it

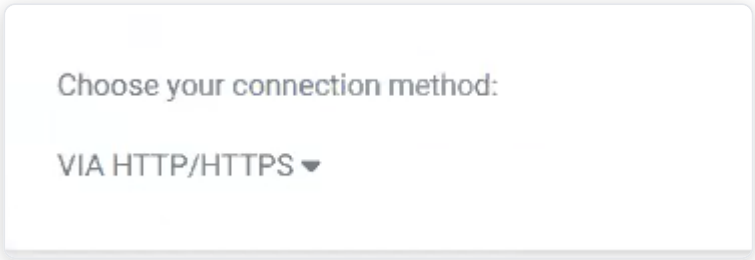

Hello
9
Code
2600ms
Code: Harness the power of Claude Code without le...

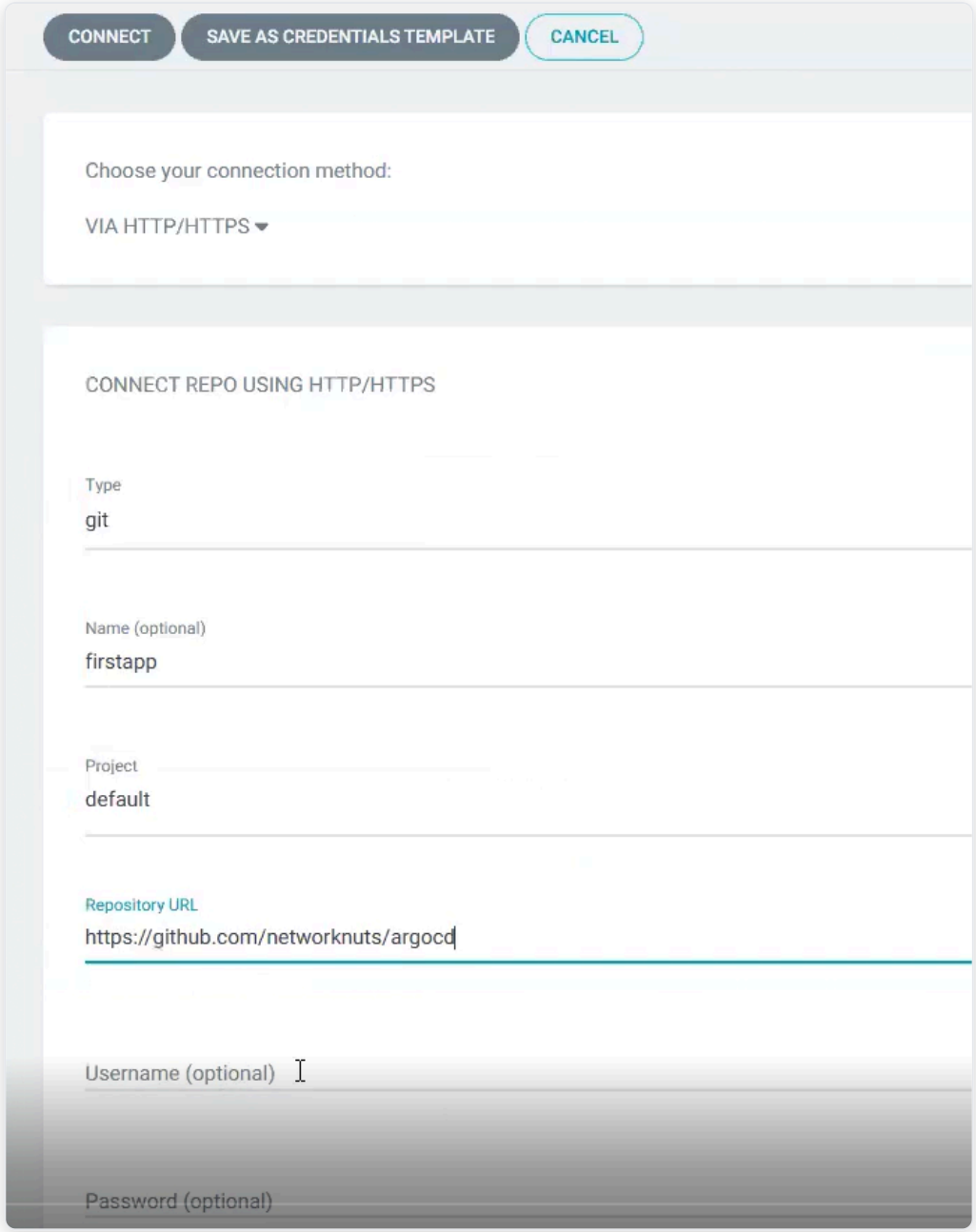

No repositories connected
Connect your repo to deploy apps.

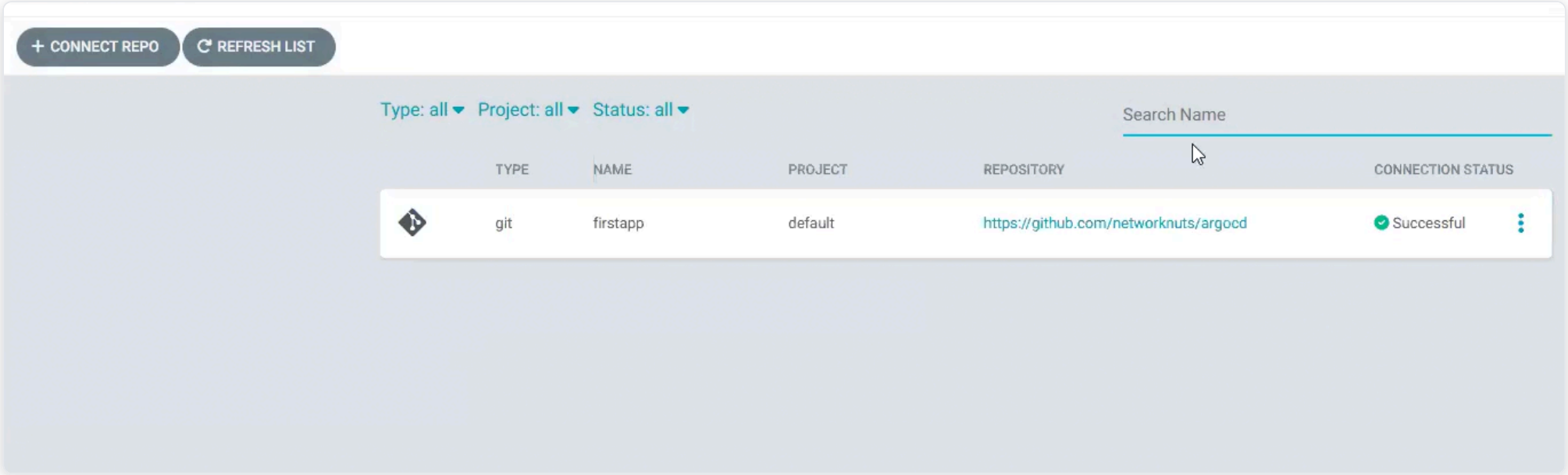

Not secure
https://localhost:8080/settings/repos
Getting Started
Imported From Fire...
New Tab
AWS Management...
Chapter 2. Brief Ove...
openstack
APN Portal
Red Hat Linux Certif...
argo
v3.5.2
Applications
ApplicationSets
Settings
User Info
Documentation
Settings / Repositories
+ CONNECT REPO
REFRESH LIST
Type: all
Project: all
Status: all

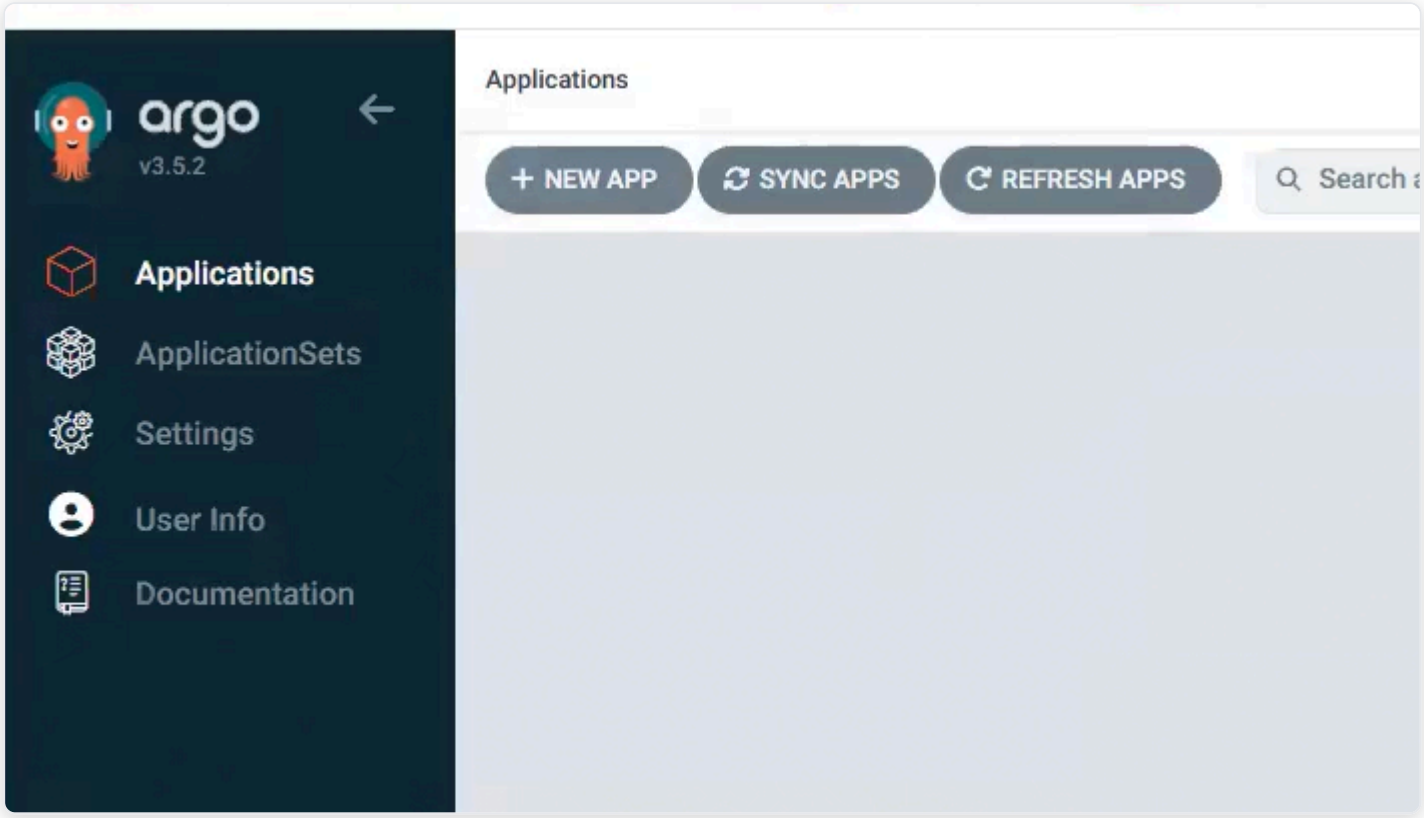

CONNECT
SAVE AS CREDENTIALS TEMPLATE
CANCEL
Choose your connection method:
VIA SSH
CONNECT REPO USING SSH
Name (mandatory for Helm)
Project
Repository URL
SSH private key data

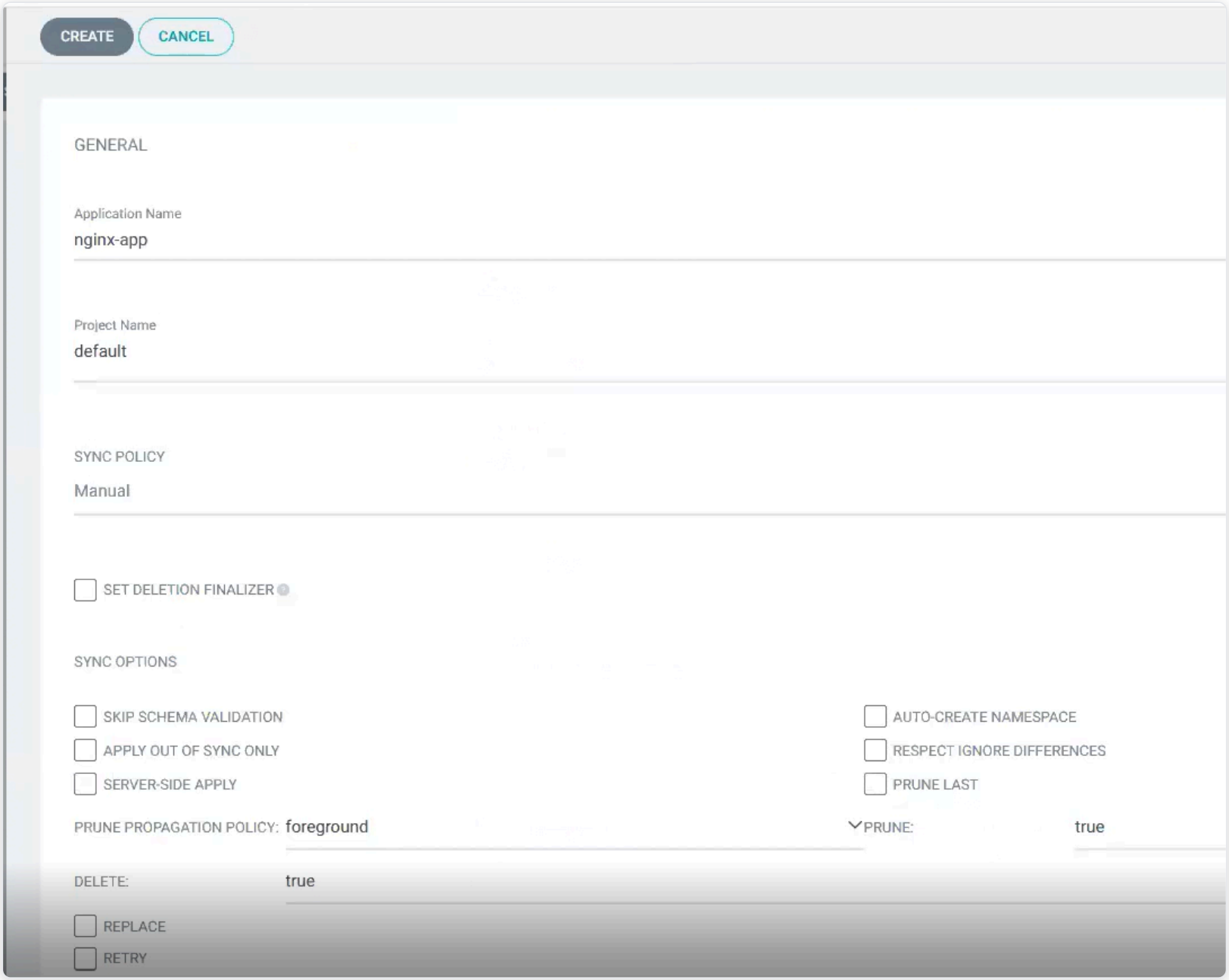

VIA SSH
VIA SSH
VIA HTTP/HTTPS
VIA GITHUB APP
VIA GOOGLE CLOUD
VIA AZURE SERVICE PRINCIPAL
Name (mandatory for Helm)


Choose your connection method:
VIA HTTP/HTTPS

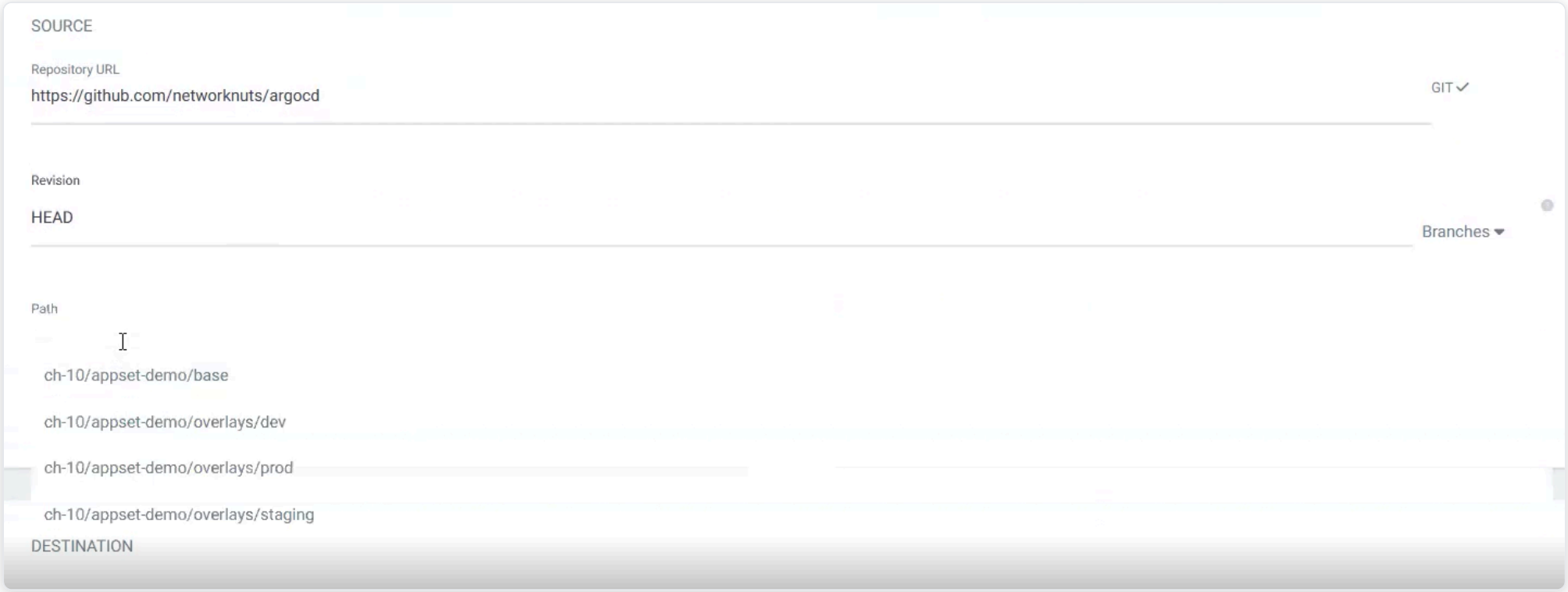

CONNECT
SAVE AS CREDENTIALS TEMPLATE
CANCEL
Choose your connection method:
VIA HTTP/HTTPS
CONNECT REPO USING HTTP/HTTPS
Type
git
Name (optional)
firstapp
Project
default
Repository URL
https://github.com/networknuts/argocd
Username (optional)
Password (optional)

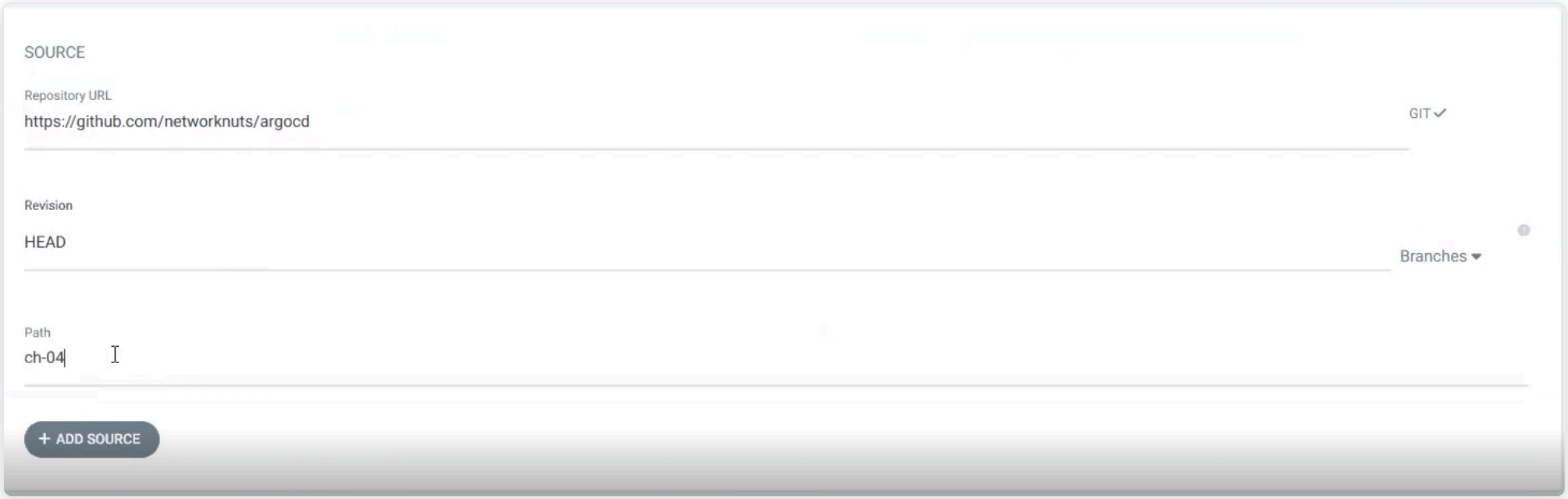

+ CONNECT REPO
REFRESH LIST
Type: all
Project: all
Status: all
Search Name
TYPE
NAME
PROJECT
REPOSITORY
CONNECTION STATUS
git
firstapp
default
https://github.com/networknuts/argocd
Successful

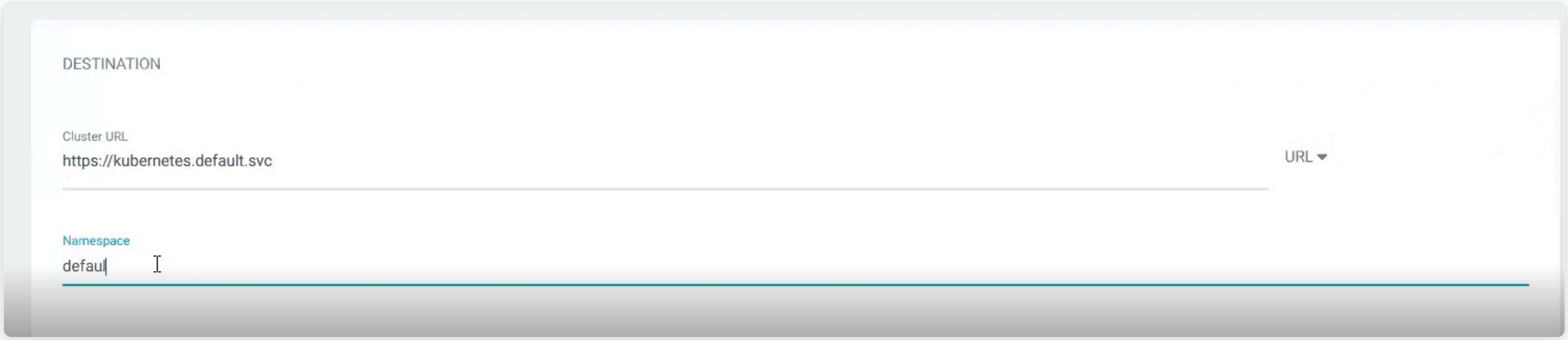

argo
v3.5.2
Applications
ApplicationSets
Settings
User Info
Documentation
Applications
+ NEW APP
SYNC APPS
REFRESH APPS
Search

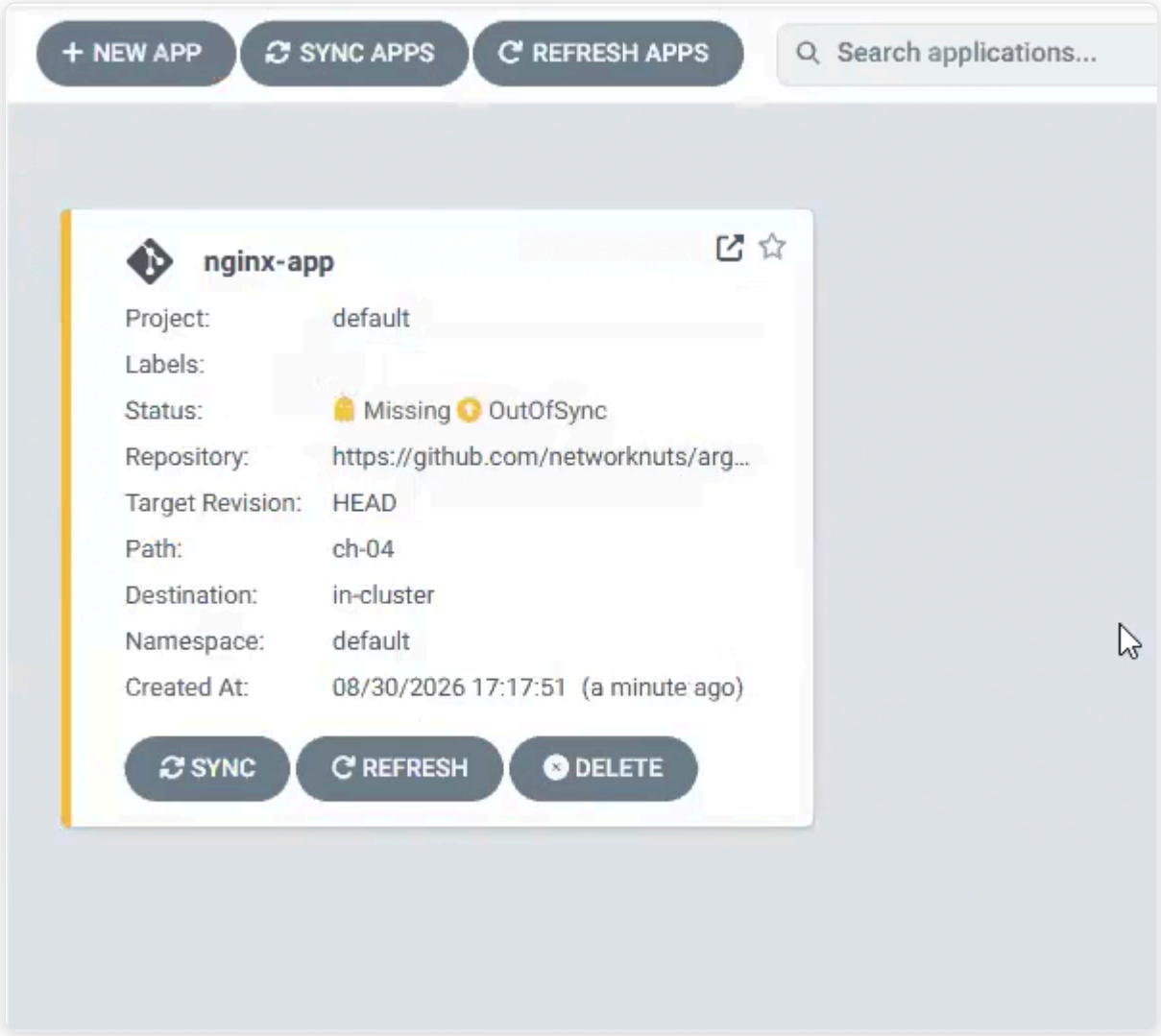

CREATE
CANCEL
GENERAL
Application Name
nginx-app
Project Name
default
SYNC POLICY
Manual
SET DELETION FINALIZER
SYNC OPTIONS
SKIP SCHEMA VALIDATION
APPLY OUT OF SYNC ONLY
SERVER-SIDE APPLY
AUTO-CREATE NAMESPACE
RESPECT IGNORE DIFFERENCES
PRUNE LAST
PRUNE PROPAGATION POLICY: foreground
PRUNE: true
DELETE: true
REPLACE
RETRY

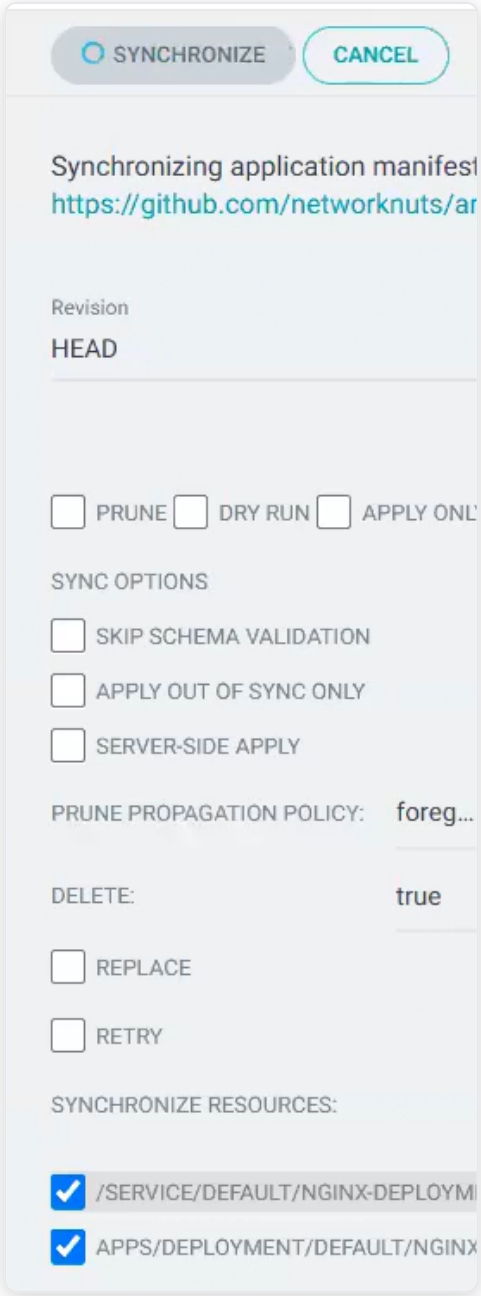

SOURCE
Repository URL
GIT
https://github.com/networkputs/argocd
Revision

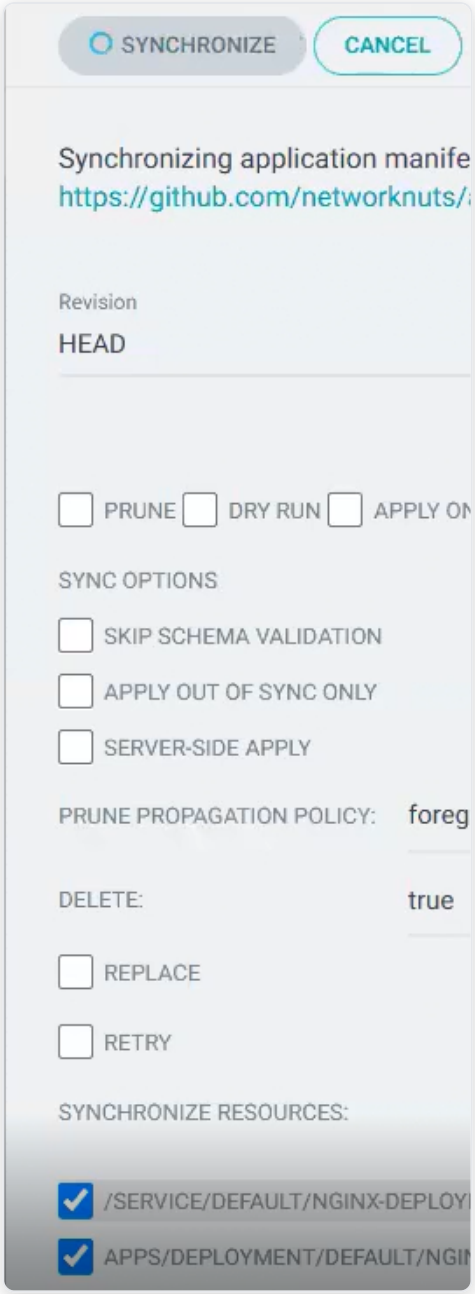

SOURCE
Repository URL
https://github.com/networknuts/argocd
GIT
Revision
HEAD
Branches
Path
ch-10/appset-demo/base
ch-10/appset-demo/overlays/dev
ch-10/appset-demo/overlays/prod
ch-10/appset-demo/overlays/staging
DESTINATION

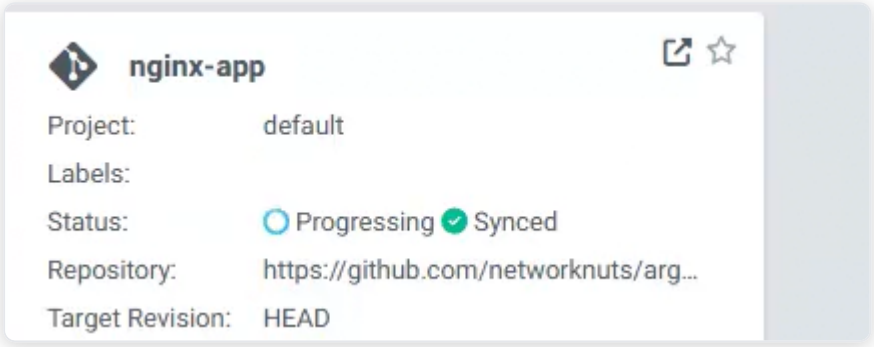

SOURCE
Repository URL
https://github.com/networknuts/argocd
GIT
Revision
HEAD
Branches
Path
ch-04
+ ADD SOURCE

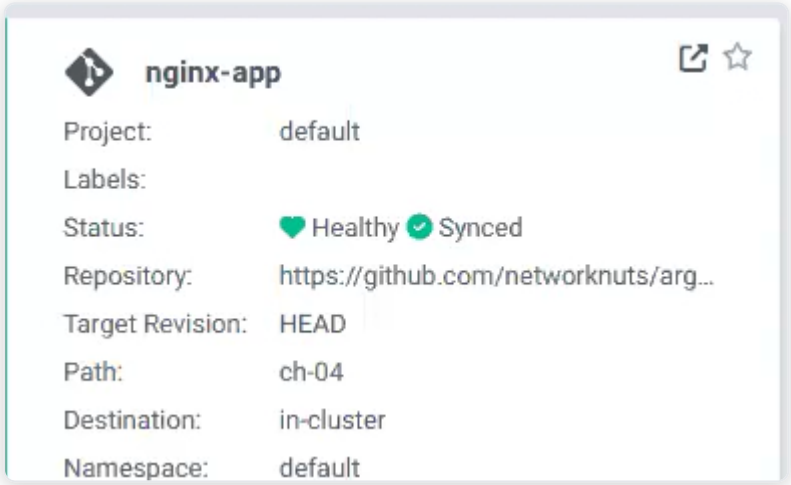

DESTINATION
Cluster URL
https://kubernetes.default.svc
URL
Namespace
defaul

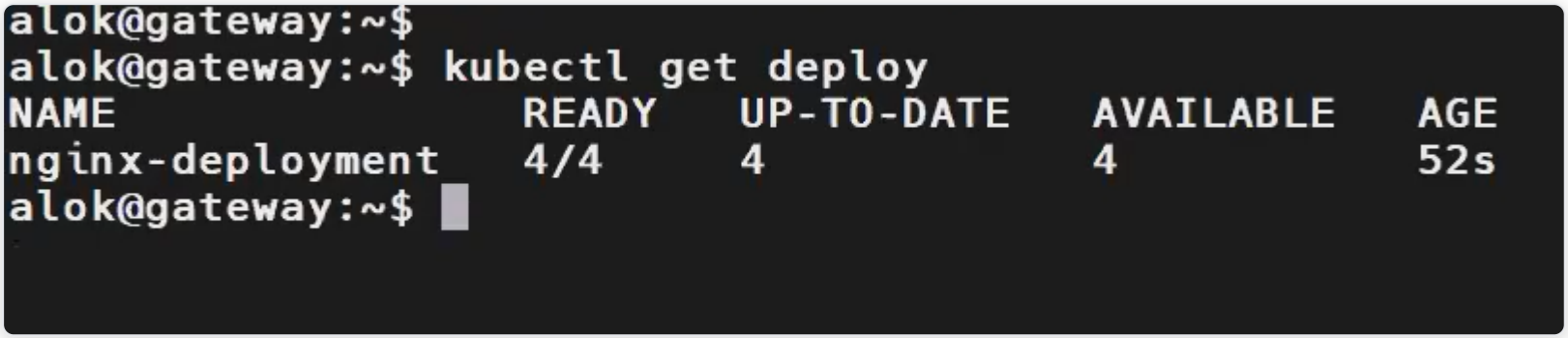

+ NEW APP
SYNC APPS
REFRESH APPS
Search applications...
nginx-app
Project: default
Labels:
Status: Missing OutOfSync
Repository: https://github.com/networknuts/arg...
Target Revision: HEAD
Path: ch-04
Destination: in-cluster
Namespace: default
Created At: 08/30/2026 17:17:51 (a minute ago)
SYNC
REFRESH
DELETE

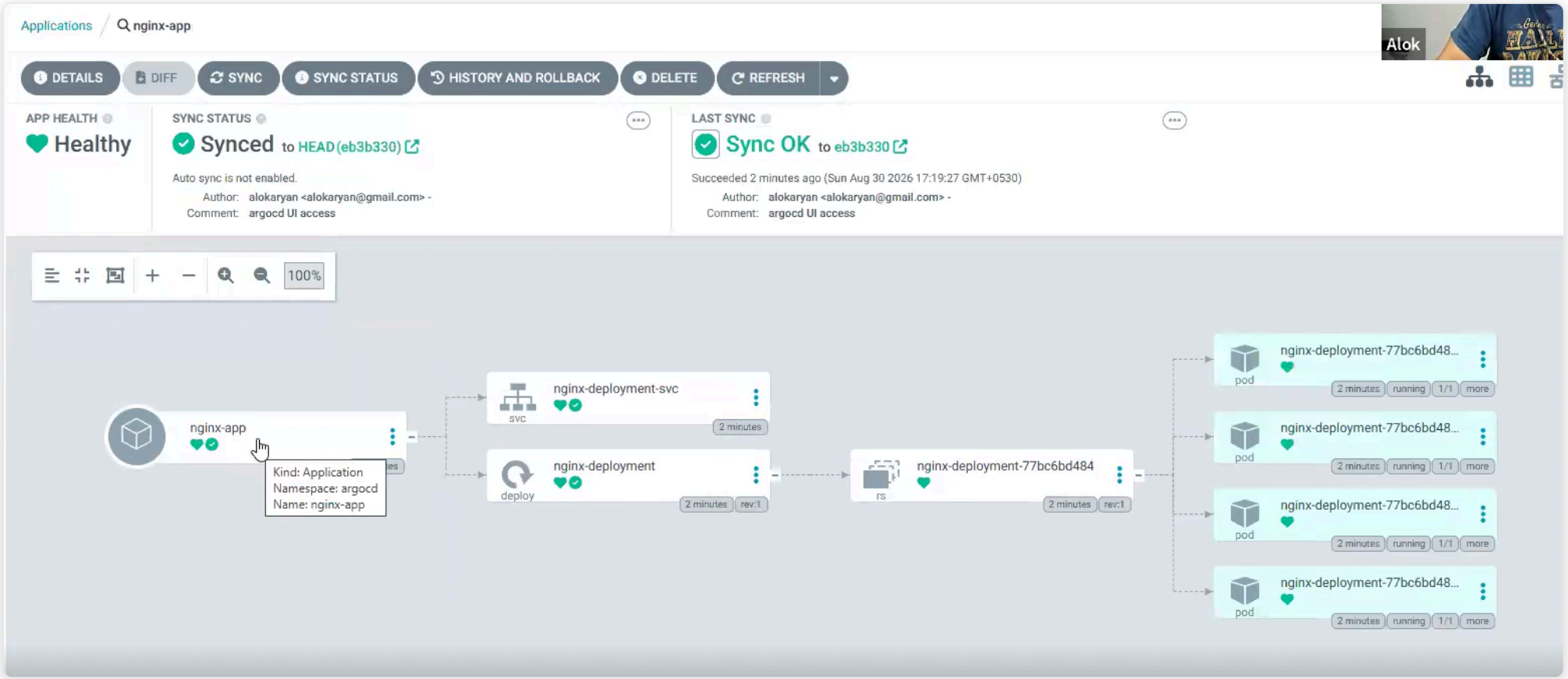

SYNCHRONIZE
CANCEL
Synchronizing application manifest
https://github.com/networknuts/ar
Revision
HEAD
PRUNE
DRY RUN
APPLY ONL
SYNC OPTIONS
SKIP SCHEMA VALIDATION
APPLY OUT OF SYNC ONLY
SERVER-SIDE APPLY
PRUNE PROPAGATION POLICY:
foreg...
DELETE:
true
REPLACE
RETRY
SYNCHRONIZE RESOURCES:
/SERVICE/DEFAULT/NGINX-DEPLOYM
APPS/DEPLOYMENT/DEFAULT/NGINX

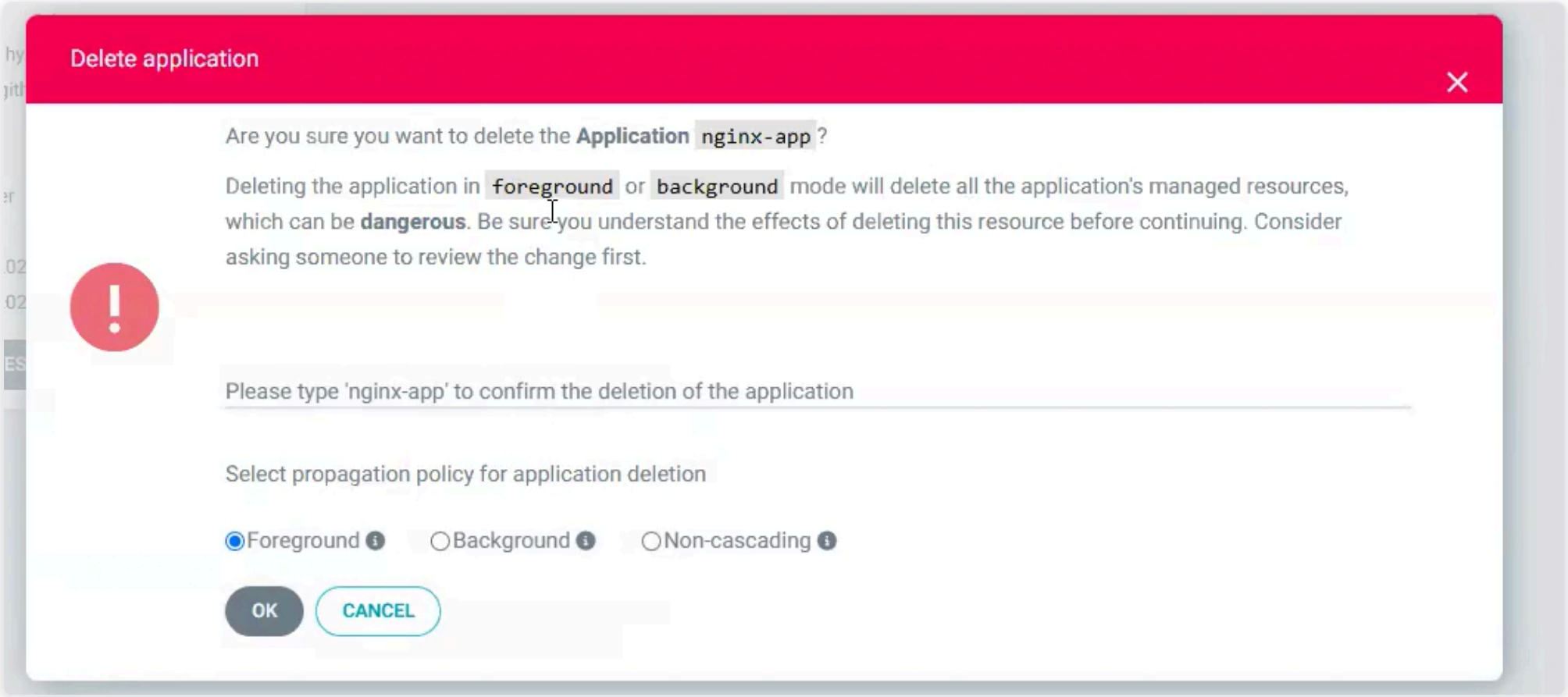

SYNCHRONIZE
CANCEL
Synchronizing application manife
https://github.com/networknuts/
Revision
HEAD
PRUNE
DRY RUN
APPLY ON
SYNC OPTIONS
SKIP SCHEMA VALIDATION
APPLY OUT OF SYNC ONLY
SERVER-SIDE APPLY
PRUNE PROPAGATION POLICY:
foreg
DELETE:
true
REPLACE
RETRY
SYNCHRONIZE RESOURCES:
/SERVICE/DEFAULT/NGINX-DEPLOY
APPS/DEPLOYMENT/DEFAULT/NGI

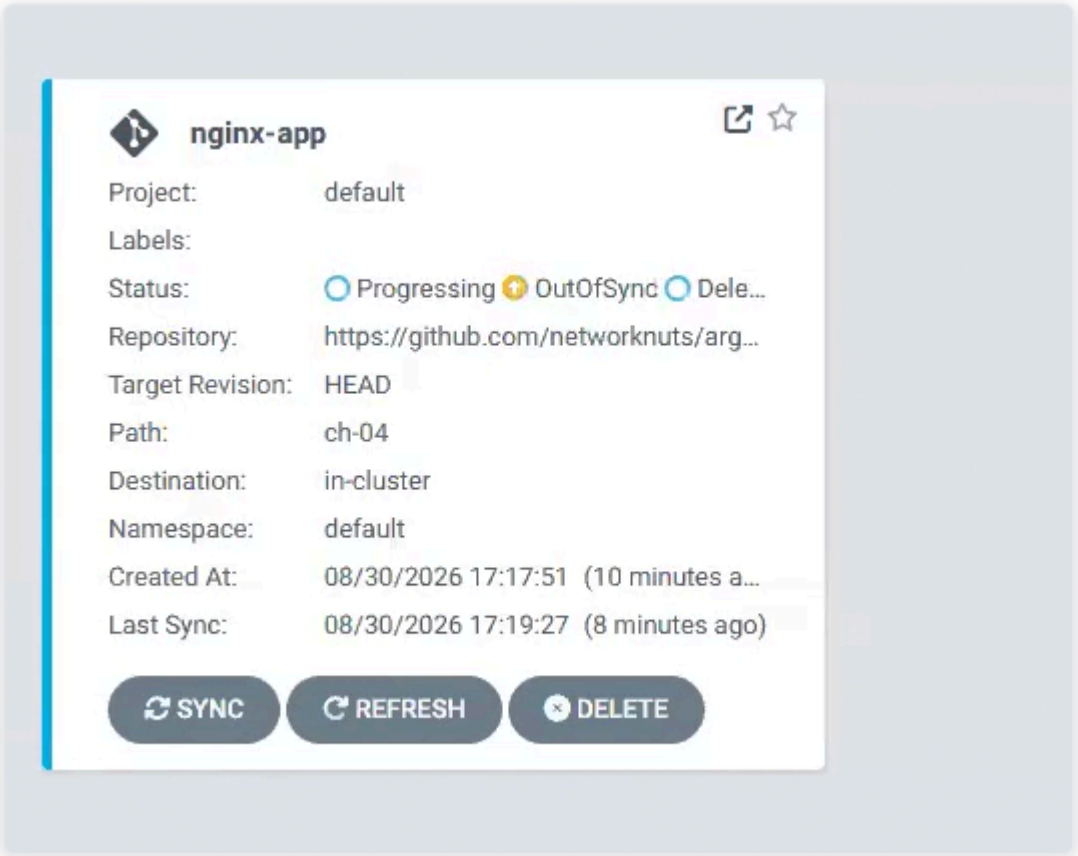

nginx-app
Project: default
Labels:
Status: Progressing Synced
Repository: https://github.com/networknuts/arg...
Target Revision: HEAD


nginx-app
Project: default
Labels:
Status: Healthy Synced
Repository: https://github.com/networknuts/arg...
Target Revision: HEAD
Path: ch-04
Destination: in-cluster
Namespace: default


alok@gateway:~$
alok@gateway:~$ kubectl get deploy
NAME READY UP-TO-DATE AVAILABLE AGE
nginx-deployment 4/4 4 4 52s
alok@gateway:~$


Applications
nginx-app
Alok
DETAILS
DIFF
SYNC
SYNC STATUS
HISTORY AND ROLLBACK
DELETE
REFRESH
APP HEALTH
Healthy
SYNC STATUS
Synced to HEAD (eb3b330)
Auto sync is not enabled.
Author: alokaryan <alokaryan@gmail.com> -
Comment: argocd UI access
LAST SYNC
Sync OK to eb3b330
Succeeded 2 minutes ago (Sun Aug 30 2026 17:19:27 GMT+0530)
Author: alokaryan <alokaryan@gmail.com> -
Comment: argocd UI access
100%
nginx-app
Kind: Application
Namespace: argocd
Name: nginx-app
svc
nginx-deployment-svc
2 minutes
deploy
nginx-deployment
2 minutes
rev:1
rs
nginx-deployment-77bc6bd484
2 minutes
rev:1
pod
nginx-deployment-77bc6bd48...
2 minutes
running
1/1
more

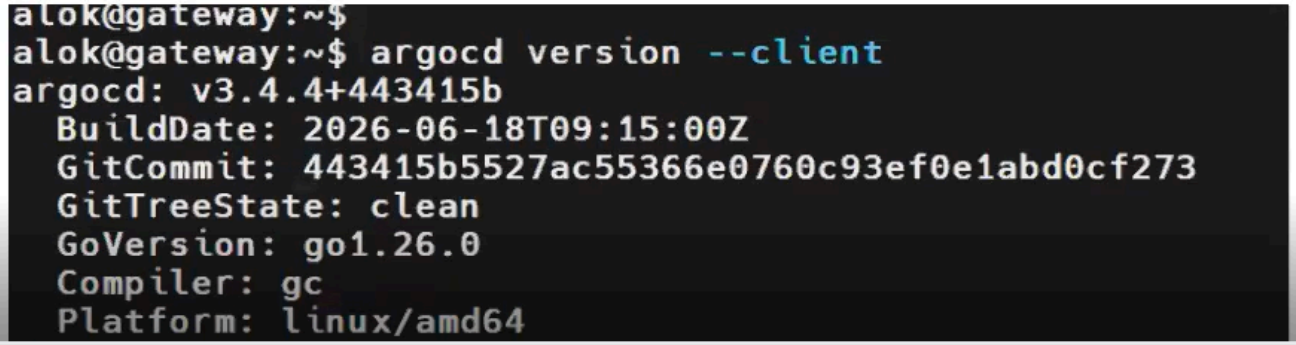

Delete application
Are you sure you want to delete the Application nginx-app ?
Deleting the application in foreground or background mode will delete all the application's managed resources, which can be dangerous. Be sure you understand the effects of deleting this resource before continuing. Consider asking someone to review the change first.
Please type 'nginx-app' to confirm the deletion of the application
Select propagation policy for application deletion
Foreground
Background
Non-cascading
OK
CANCEL

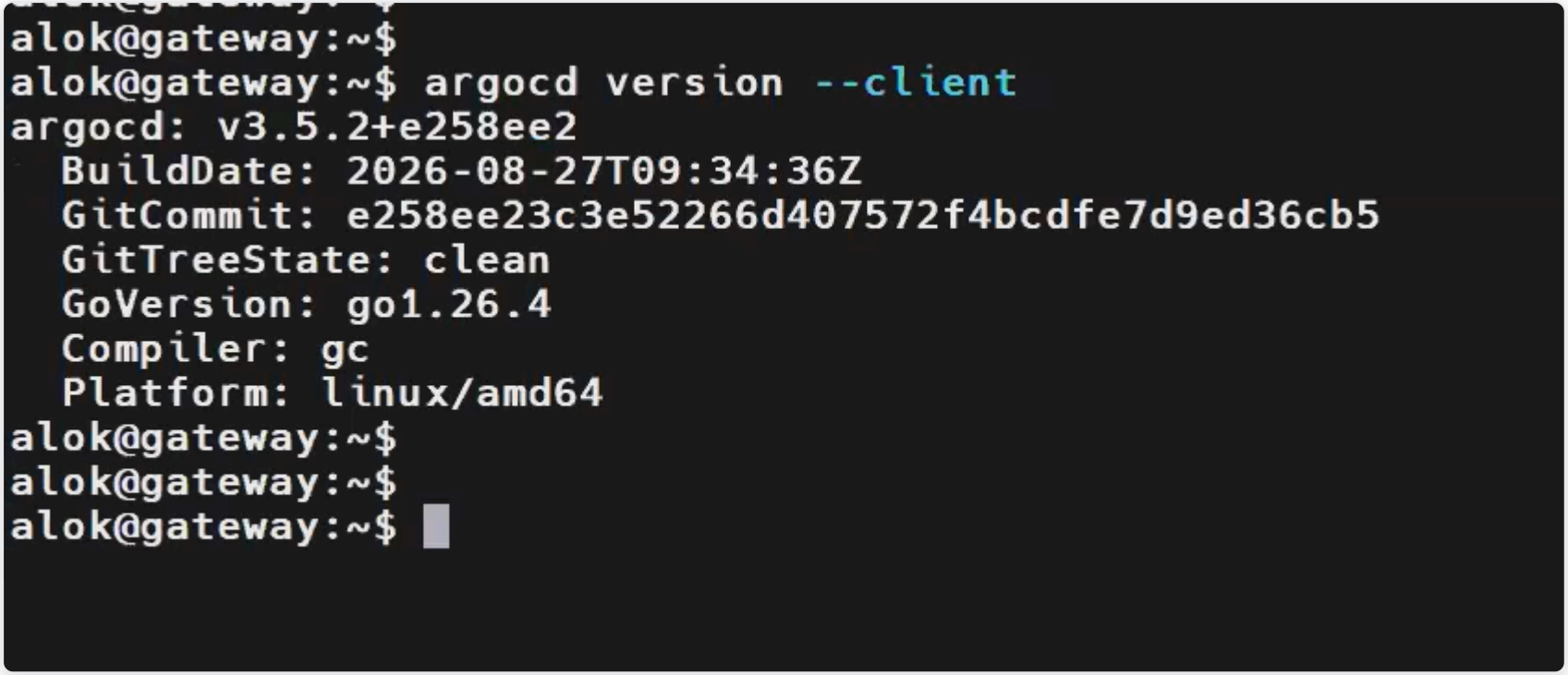

nginx-app
Project: default
Labels:
Status: Progressing OutOfSync Dele...
Repository: https://github.com/networknuts/arg...
Target Revision: HEAD
Path: ch-04
Destination: in-cluster
Namespace: default
Created At: 08/30/2026 17:17:51 (10 minutes a...
Last Sync: 08/30/2026 17:19:27 (8 minutes ago)
SYNC
REFRESH
DELETE

Installing Argo CD CLI

The Argo CD CLI (argocd) gives you full control over your GitOps workflows from the terminal. It's a great tool f
automation and for users who prefer command-line tools over the UI.

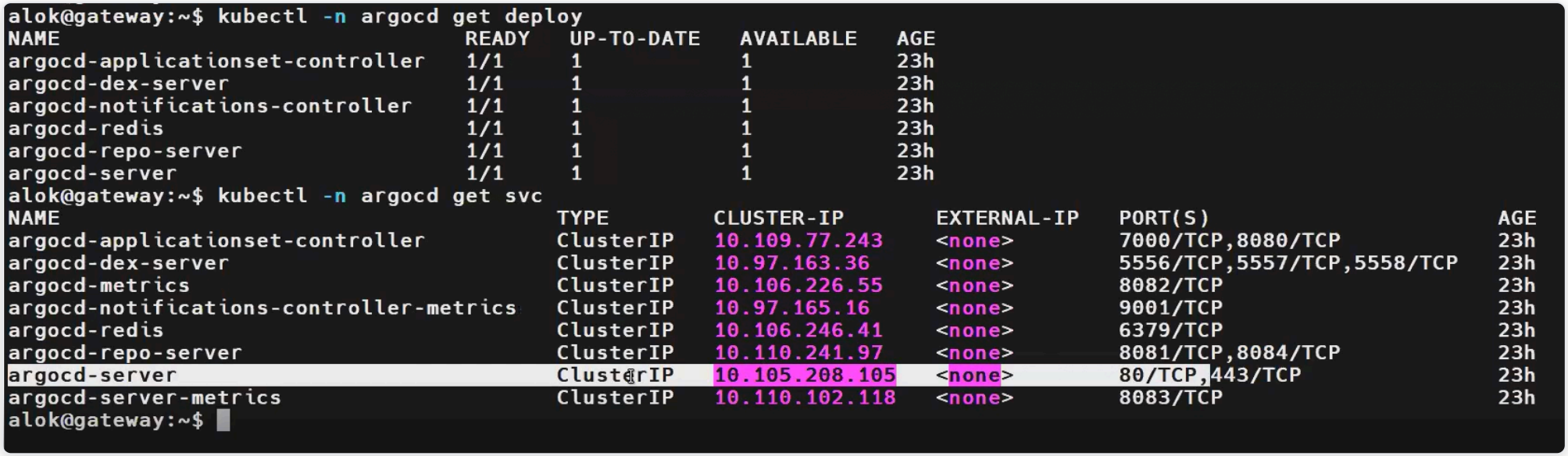
```
# 1. Download the latest binary
curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-am
# 2. Install it with executable permissions to your system path
sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
# 3. Clean up the downloaded installer file
rm argocd-linux-amd64
# 4. Verify
argocd version --client
```




alok@gateway:~$
alok@gateway:~$ argocd version --client
argocd: v3.5.2+e258ee2
BuildDate: 2026-08-27T09:34:36Z
GitCommit: e258ee23c3e52266d407572f4bcdfe7d9ed36cb5
GitTreeState: clean
GoVersion: go1.26.4
Compiler: gc
Platform: linux/amd64
alok@gateway:~$
alok@gateway:~$
alok@gateway:~$

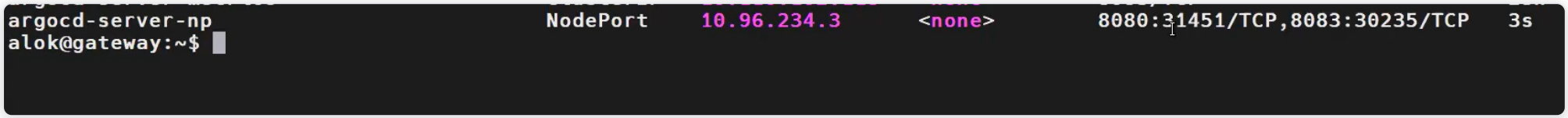
We will be using our Bastion Host to access Kubernetes Cluster which has Argo CD installed. The easiest way here to to expose argocd-server deployment as "NodePort" service and then use NodeIP:PORT to access Argo CD using Argo CD CLI.

```
alok@gateway:~$
alok@gateway:~$ kubectl -n argocd get deploy
NAME                               READY   UP-TO-DATE   AVAILABLE   AGE
argocd-applicationset-controller   1/1     1            1           4d
argocd-dex-server                  1/1     1            1           4d
argocd-notifications-controller    1/1     1            1           4d
argocd-redis                       1/1     1            1           4d
argocd-repo-server                 1/1     1            1           4d
argocd-server                      1/1     1            1           4d
alok@gateway:~$
alok@gateway:~$ kubectl -n argocd expose deployment argocd-server \
> --name argocd-server-np --type=NodePort
service/argocd-server-np exposed
alok@gateway:~$
alok@gateway:~$ kubectl -n argocd get svc
NAME                                TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
argocd-applicationset-controller    ClusterIP   10.101.110.191   <none>        7000/TCP,8080/TCP            4d
argocd-dex-server                   ClusterIP   10.103.69.191    <none>        5556/TCP,5557/TCP,5558/TCP   4d
argocd-metrics                      ClusterIP   10.100.23.149    <none>        8082/TCP                     4d
argocd-notifications-controller-metrics   ClusterIP   10.103.217.214   <none>   9001/TCP                     4d
argocd-redis                        ClusterIP   10.109.125.94    <none>        6379/TCP                     4d
argocd-repo-server                  ClusterIP   10.104.251.185   <none>        8081/TCP,8084/TCP            4d
argocd-server                       ClusterIP   10.107.167.109   <none>        80/TCP,443/TCP               4d
argocd-server-metrics               ClusterIP   10.101.233.255   <none>        8083/TCP                     4d
argocd-server-np                    NodePort    10.96.102.186    <none>        8080:31530/TCP,8083:32517/TCP   3s
alok@gateway:~$
```


alok@gateway:~$ kubectl -n argocd get deploy
NAME READY UP-TO-DATE AVAILABLE AGE
argocd-applicationset-controller 1/1 1 1 23h
argocd-dex-server 1/1 1 1 23h
argocd-notifications-controller 1/1 1 1 23h
argocd-redis 1/1 1 1 23h
argocd-repo-server 1/1 1 1 23h
argocd-server 1/1 1 1 23h
alok@gateway:~$ kubectl -n argocd get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
argocd-applicationset-controller ClusterIP 10.109.77.243 <none> 7000/TCP,8080/TCP 23h
argocd-dex-server ClusterIP 10.97.163.36 <none> 5556/TCP,5557/TCP,5558/TCP 23h
argocd-metrics ClusterIP 10.106.226.55 <none> 8082/TCP 23h
argocd-notifications-controller-metrics ClusterIP 10.97.165.16 <none> 9001/TCP 23h
argocd-redis ClusterIP 10.106.246.41 <none> 6379/TCP 23h
argocd-repo-server ClusterIP 10.110.241.97 <none> 8081/TCP,8084/TCP 23h
argocd-server ClusterIP 10.105.208.105 <none> 80/TCP,443/TCP 23h
argocd-server-metrics ClusterIP 10.110.102.118 <none> 8083/TCP 23h
alok@gateway:~$


argocd-applicationset-controller ClusterIP 10.109.77.243 <none> 7000/TCP,8080/TCP
argocd-dex-server ClusterIP 10.97.163.36 <none> 5556/TCP,5557/TCP,5558/TCP
argocd-metrics ClusterIP 10.106.226.55 <none> 8082/TCP
argocd-notifications-controller-metrics ClusterIP 10.97.165.16 <none> 9001/TCP
argocd-redis ClusterIP 10.106.246.41 <none> 6379/TCP
argocd-repo-server ClusterIP 10.110.241.97 <none> 8081/TCP,8084/TCP
argocd-server ClusterIP 10.105.208.105 <none> 80/TCP,443/TCP
argocd-server-metrics ClusterIP 10.110.102.118 <none> 8083/TCP
CD GitOps with Kubernetes 0500PM - 0630PM IST Weekend 23082026-0-1700-1(30-Aug-2026-5-00 PM-6-30 PM
alok@gateway:~$ kubectl -n argocd expose deploy argocd-server --name argocd-server-np --type NodePort
monitoring

```
alok@gateway:~$
alok@gateway:~$ kubectl -n argocd expose deploy argocd-server --name argocd-server-np --type NodePort
```


argocd-server-np NodePort 10.96.234.3 <none> 8080:31451/TCP,8083:30235/TCP 3s
alok@gateway:~$

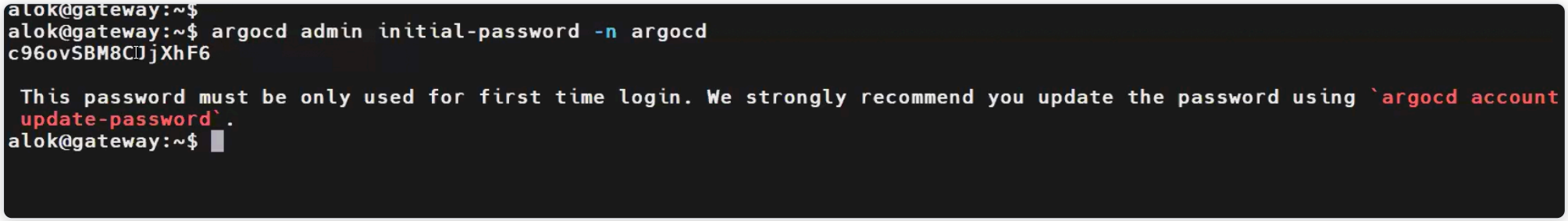
#1 - Get Argo CD admin password.
argocd admin initial-password -n argocd

```
alok@gateway:~$
alok@gateway:~$ argocd admin initial-password -n argocd
kzFS1lfzgwAVJdty

 This password must be only used for first time login. We strongly r
 update-password`.
```

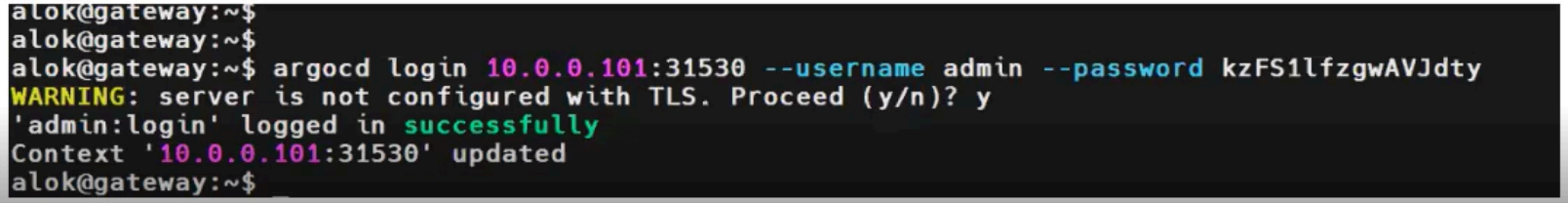
#2 - Login using:
argocd login 10.0.0.101:31530 --username admin --password kzFS1lfzgwAVJdty

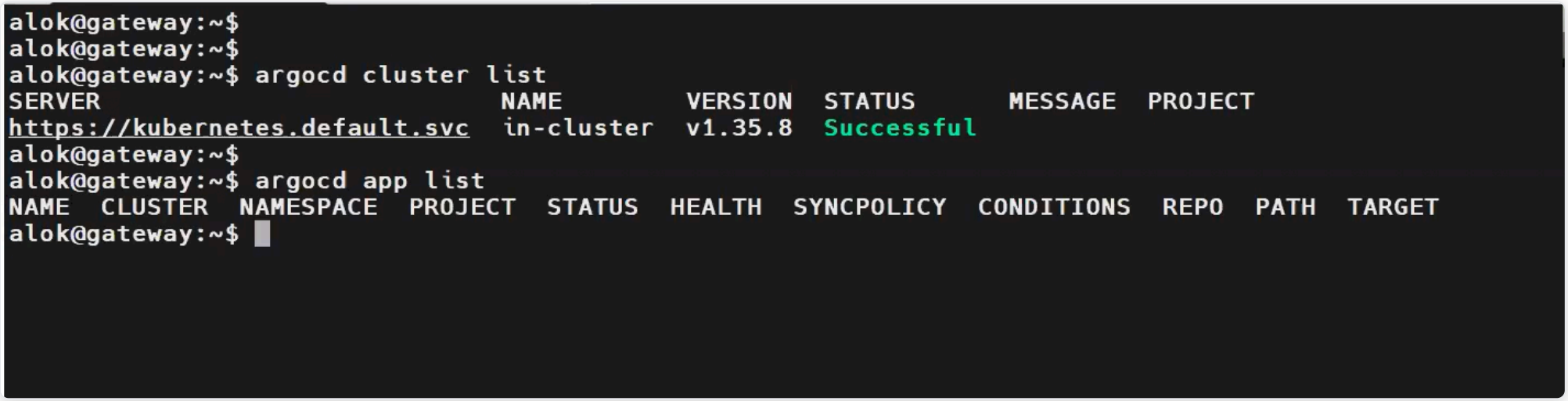
```
alok@gateway:~$
alok@gateway:~$
alok@gateway:~$ argocd login 10.0.0.101:31530 --username admin --password kzFS1lfzgwAVJdty
WARNING: server is not configured with TLS. Proceed (y/n)? y
'admin:login' logged in successfully
Context '10.0.0.101:31530' updated
alok@gateway:~$
```

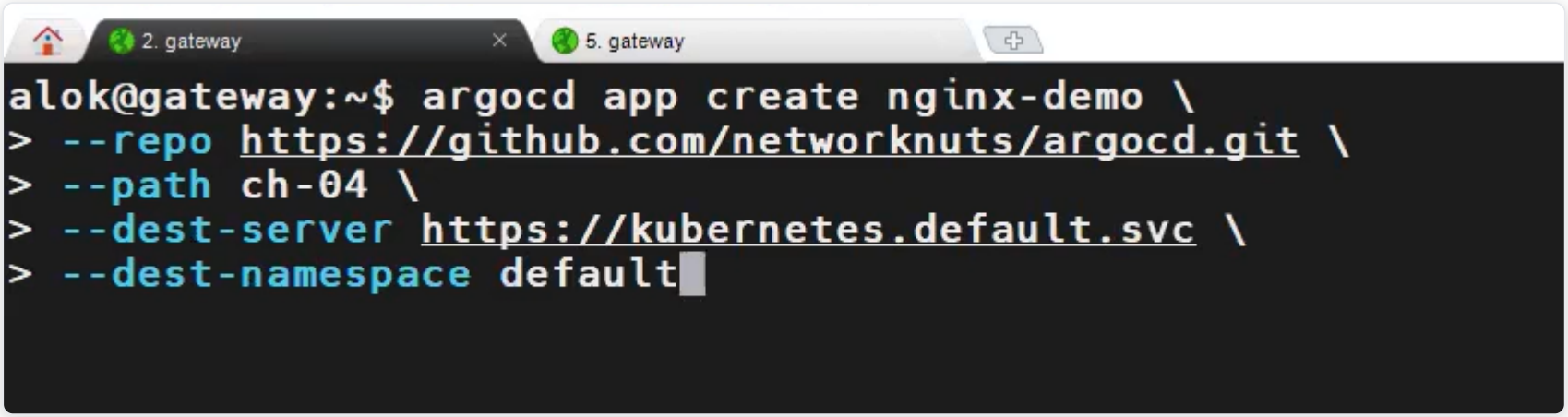

alok@gateway:~$
alok@gateway:~$ argocd admin initial-password -n argocd
c96ovSBM8C jXhF6
This password must be only used for first time login. We strongly recommend you update the password using `argocd account update-password`.
alok@gateway:~$

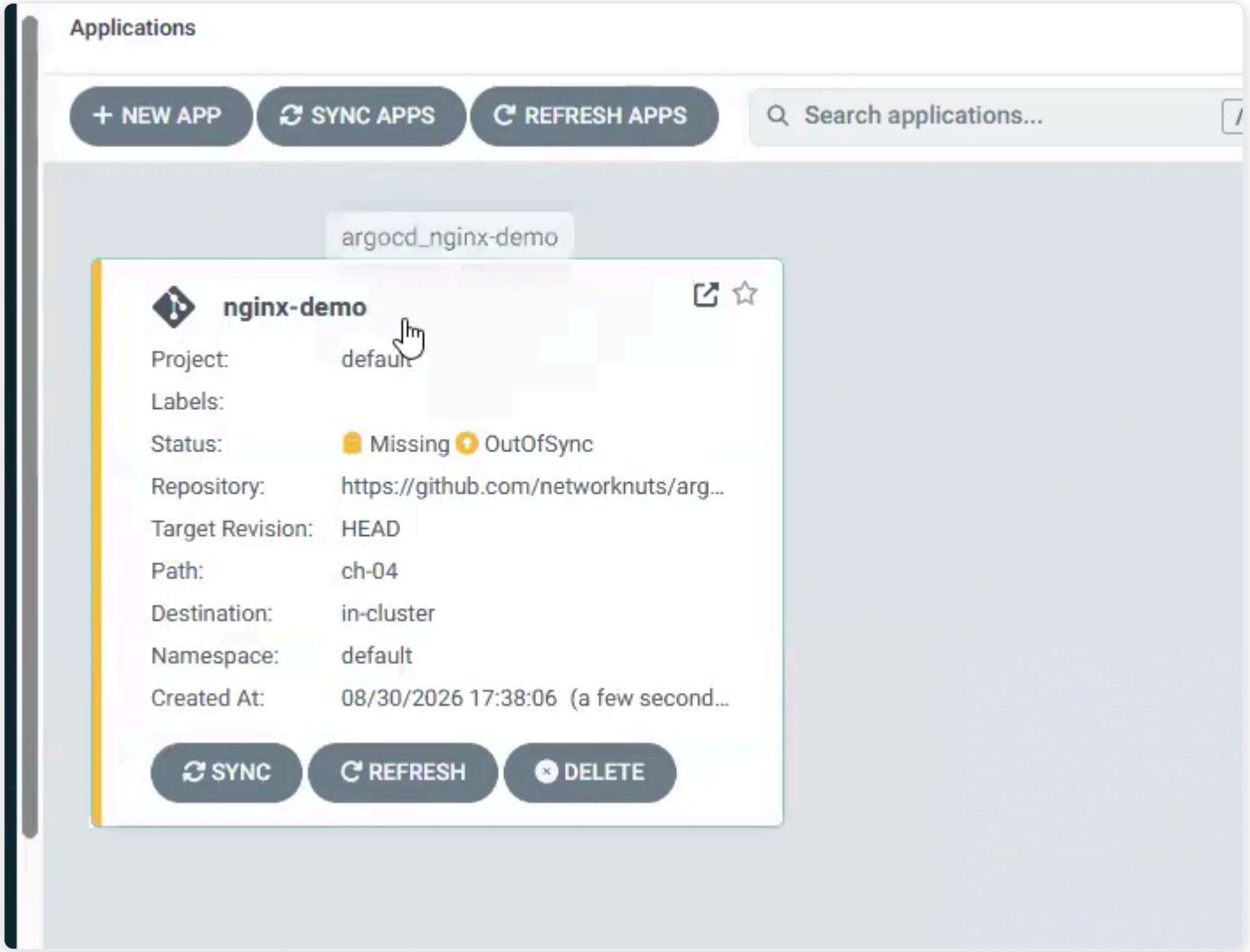
#2 - Login using:

```
argocd login 10.0.0.101:31530 --username admin --password kzFS1lfzgwAVJdty
```




alok@gateway:~$
alok@gateway:~$
alok@gateway:~$ argocd cluster list
SERVER NAME VERSION STATUS MESSAGE PROJECT
https://kubernetes.default.svc in-cluster v1.35.8 Successful
alok@gateway:~$
alok@gateway:~$ argocd app list
NAME CLUSTER NAMESPACE PROJECT STATUS HEALTH SYNCPOLICY CONDITIONS REPO PATH TARGET
alok@gateway:~$


2. gateway
5. gateway
alok@gateway:~$ argocd app create nginx-demo \
> --repo https://github.com/networknuts/argocd.git \
> --path ch-04 \
> --dest-server https://kubernetes.default.svc \
> --dest-namespace default


Applications
+ NEW APP
SYNC APPS
REFRESH APPS
Search applications...
argocd_nginx-demo
nginx-demo
Project: default
Labels:
Status: Missing OutOfSync
Repository: https://github.com/networknuts/arg...
Target Revision: HEAD
Path: ch-04
Destination: in-cluster
Namespace: default
Created At: 08/30/2026 17:38:06 (a few second...
SYNC
REFRESH
DELETE

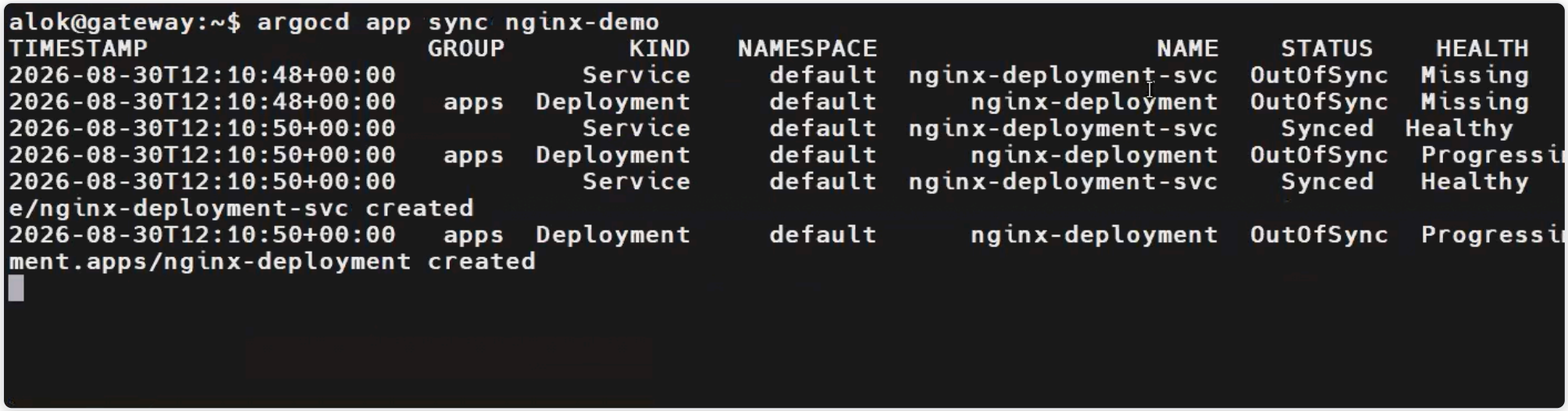
```
alok@gateway:~$ argocd app list
NAME               CLUSTER                         NAMESPACE  PROJECT  STATUS     HEALTH   SYNCPOLICY  CONDITIONS  REPO
                                  PATH   TARGET
argocd/nginx-demo  https://kubernetes.default.svc  default    default  OutOfSync  Missing  Manual      <none>      https:/
/github.com/networknuts/argocd.git  ch-04
alok@gateway:~$
alok@gateway:~$
alok@gateway:~$ argocd app get nginx-demo
Name:               argocd/nginx-demo
Project:            default
Server:             https://kubernetes.default.svc
Namespace:          default
URL:                https://10.0.0.101:31451/applications/nginx-demo
Source:
- Repo:             https://github.com/networknuts/argocd.git
  Target:
  Path:             ch-04
SyncWindow:         Sync Allowed
Sync Policy:        Manual
Sync Status:        OutOfSync from  (eb3b330)
Health Status:      Missing

GROUP  KIND        NAMESPACE  NAME                  STATUS     HEALTH   HOOK  MESSAGE
       Service     default    nginx-deployment-svc  OutOfSync  Missing
apps   Deployment  default    nginx-deployment      OutOfSync  Missing
alok@gateway:~$
```

Now "sync" the application using CLI

argocd app sync nginx-demo

```
alok@gateway:~$
alok@gateway:~$ argocd app sync nginx-demo
TIMESTAMP                  GROUP        KIND   NAMESPACE                  NAME
2026-07-01T16:30:21+00:00            Service     default  nginx-deployment-svc
2026-07-01T16:30:21+00:00   apps  Deployment     default      nginx-deployment
2026-07-01T16:30:22+00:00            Service     default  nginx-deployment-svc
inx-deployment-svc configured
2026-07-01T16:30:22+00:00   apps  Deployment     default      nginx-deployment
.apps/nginx-deployment configured
2026-07-01T16:30:22+00:00            Service     default  nginx-deployment-svc
nx-deployment-svc configured
```


alok@gateway:~$ argocd app sync nginx-demo
TIMESTAMP GROUP KIND NAMESPACE NAME STATUS HEALTH
2026-08-30T12:10:48+00:00 Service default nginx-deployment-svc OutOfSync Missing
2026-08-30T12:10:48+00:00 apps Deployment default nginx-deployment OutOfSync Missing
2026-08-30T12:10:50+00:00 Service default nginx-deployment-svc Synced Healthy
2026-08-30T12:10:50+00:00 apps Deployment default nginx-deployment OutOfSync Progressi
2026-08-30T12:10:50+00:00 Service default nginx-deployment-svc Synced Healthy
e/nginx-deployment-svc created
2026-08-30T12:10:50+00:00 apps Deployment default nginx-deployment OutOfSync Progressi
ment.apps/nginx-deployment created

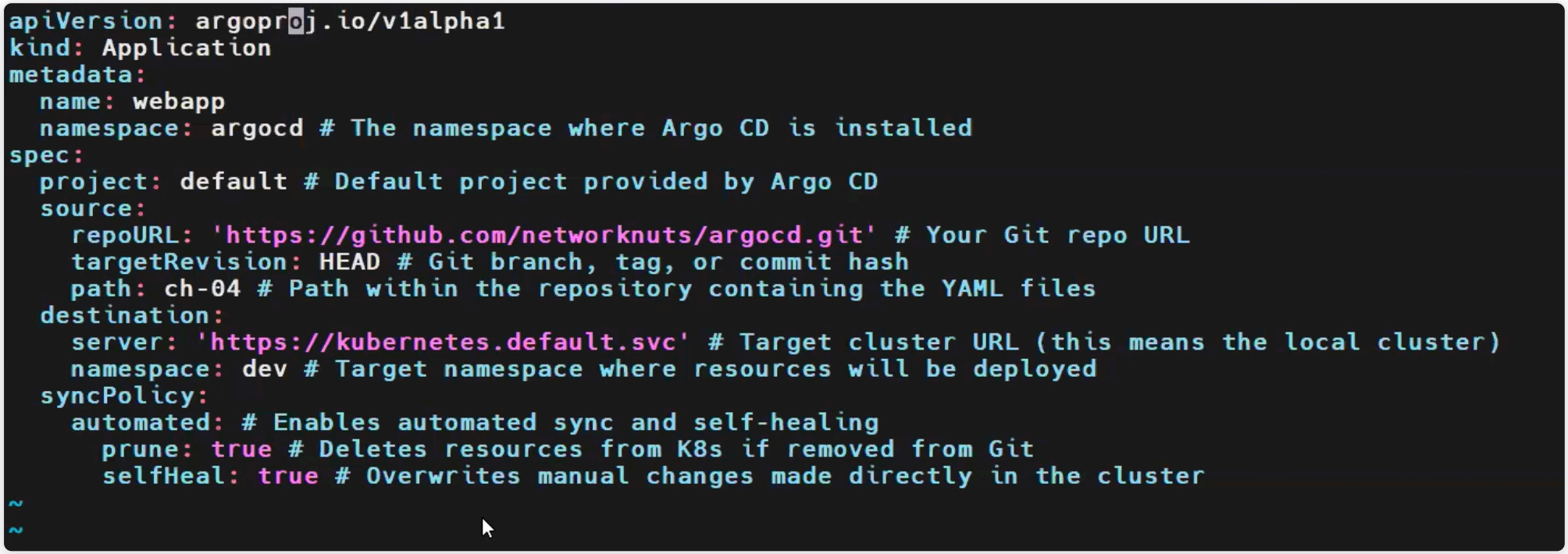
```
Name:               argocd/nginx-demo
Project:            default
Server:             https://kubernetes.default.svc
Namespace:          default
URL:                https://10.0.0.101:31451/applications/nginx-demo
Source:
- Repo:             https://github.com/networknuts/argocd.git
  Target:
  Path:             ch-04
SyncWindow:         Sync Allowed
Sync Policy:        Manual
Sync Status:        Synced to  (eb3b330)
Health Status:      Progressing

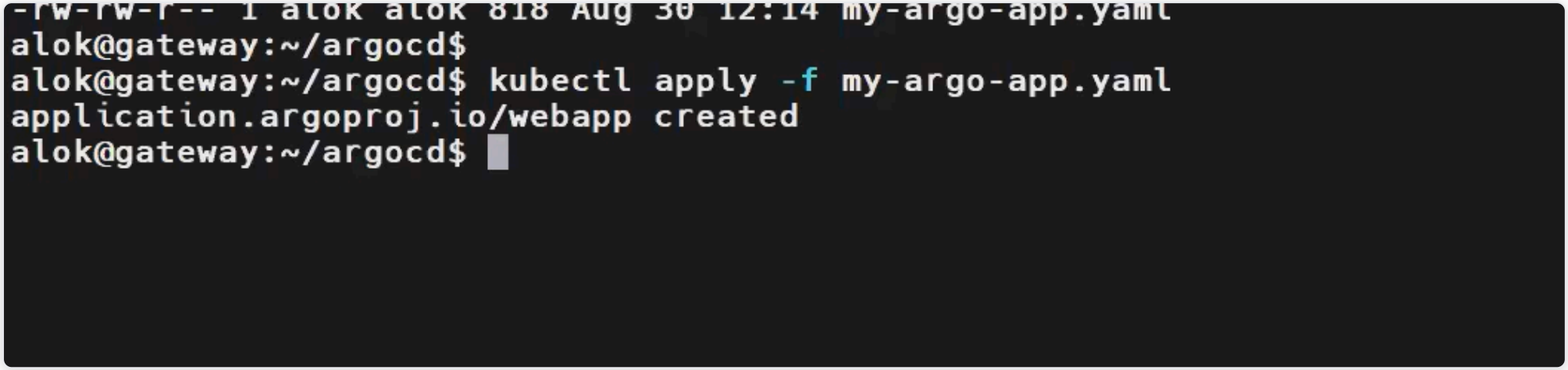
Operation:          Sync
Sync Revision:      eb3b330992004fb6846a253d024d3336ad0ee762
Phase:              Succeeded
Start:              2026-08-30 12:10:48 +0000 UTC
Finished:           2026-08-30 12:10:50 +0000 UTC
Duration:           2s
Message:            successfully synced (all tasks run)
```

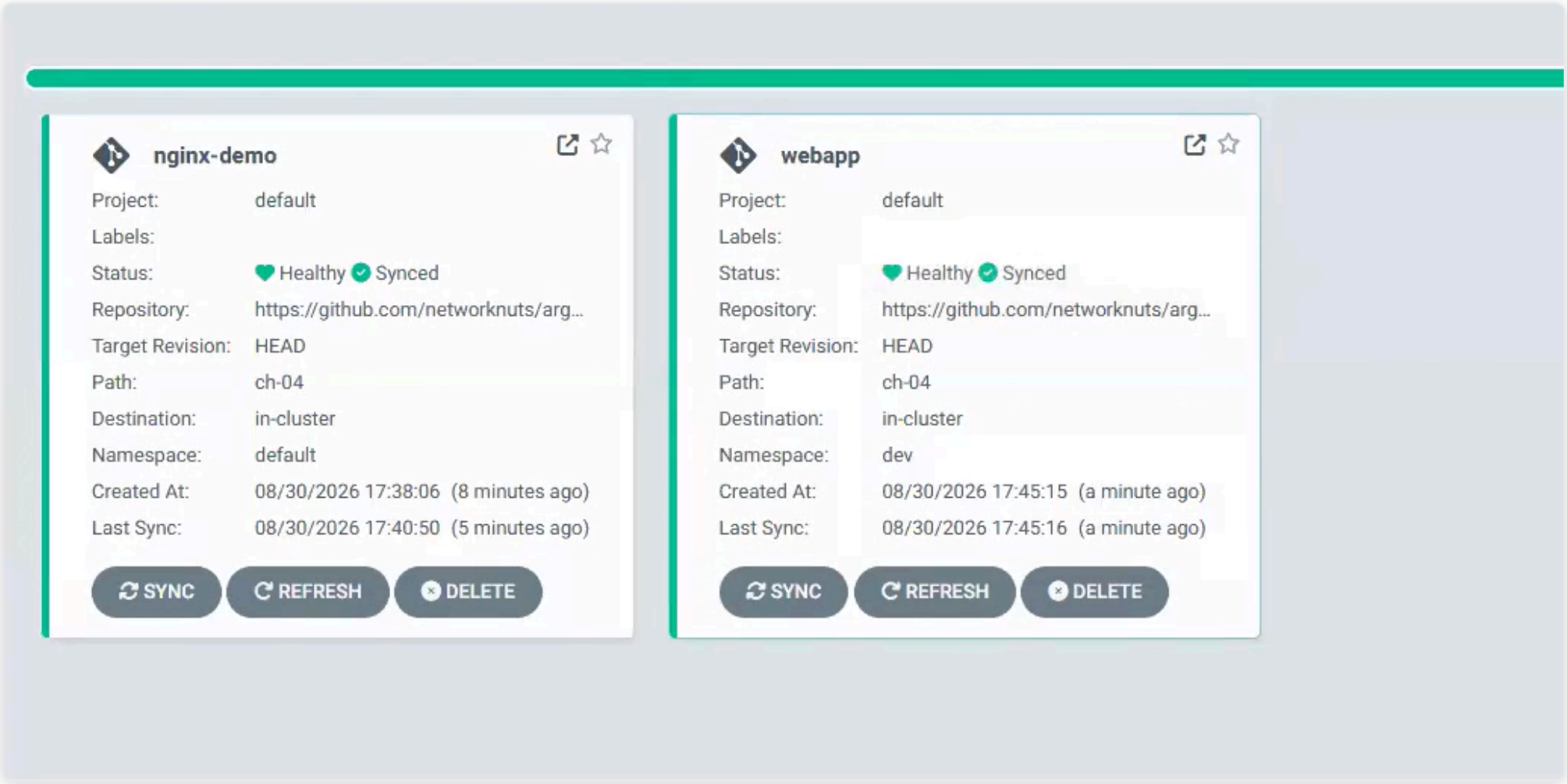
You can create a separate resource file for the app and deploy it with kubectl, like any other resource.

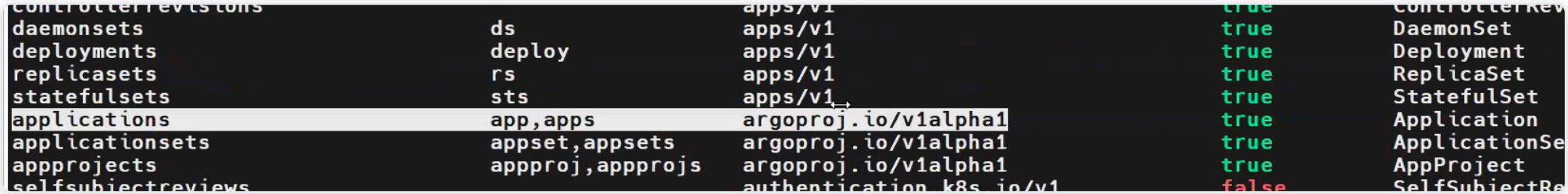
vim argo-application.yaml

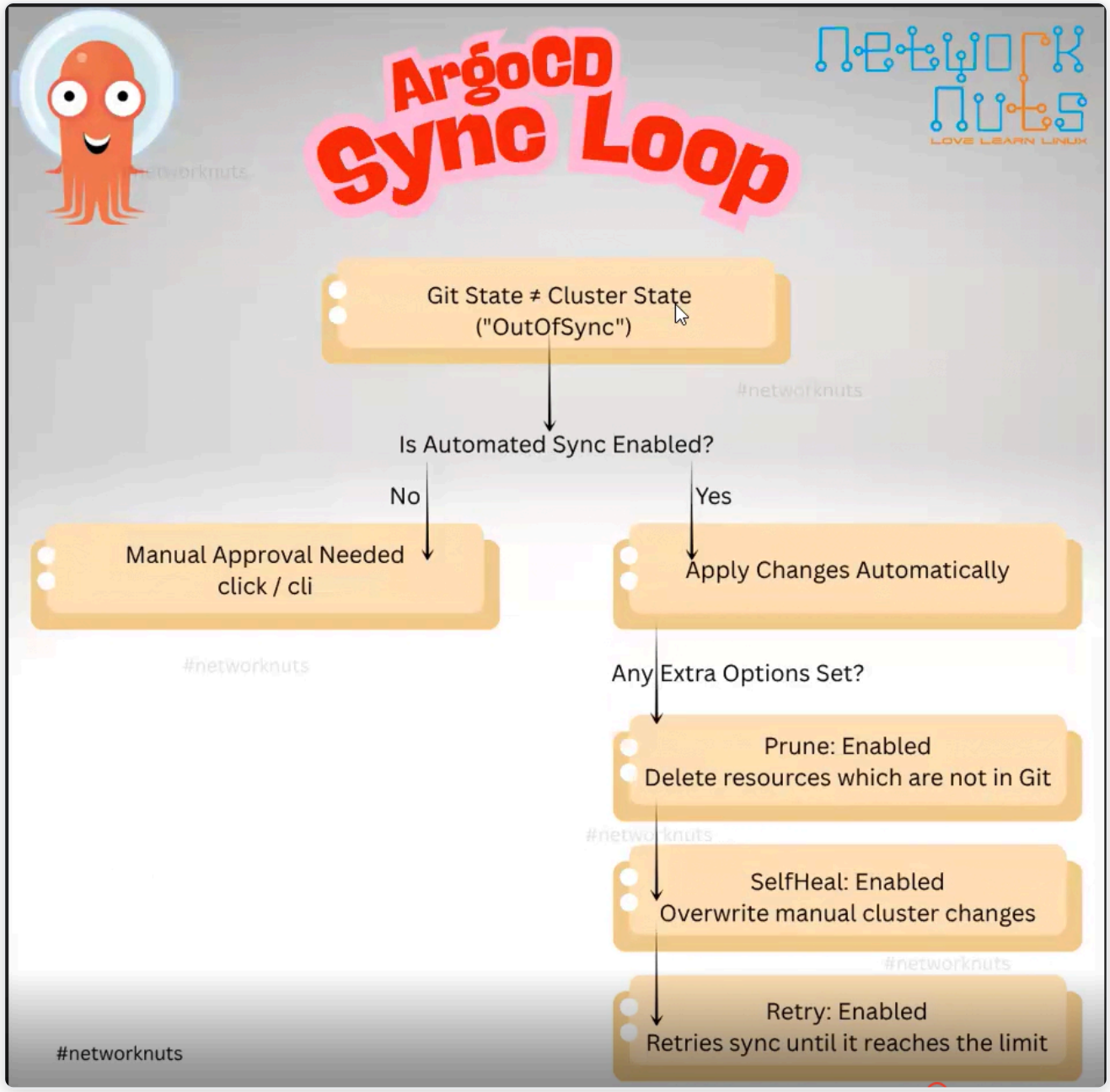
```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: webapp
  namespace: argocd # The namespace where Argo CD is installed
spec:
  project: default # Default project provided by Argo CD
  source:
    repoURL: 'https://github.com/networknuts/argocd.git' # Your Git repo URL
    targetRevision: HEAD # Git branch, tag, or commit hash
    path: ch-04 # Path within the repository containing the YAML files
  destination:
    server: 'https://kubernetes.default.svc' # Target cluster URL (this means the local cluster)
    namespace: dev # Target namespace where resources will be deployed
  syncPolicy:
    automated: # Enables automated sync and self-healing
      prune: true # Deletes resources from K8s if removed from Git
      selfHeal: true # Overwrites manual changes made directly in the cluster
```


apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
name: webapp
namespace: argocd # The namespace where Argo CD is installed
spec:
project: default # Default project provided by Argo CD
source:
repoURL: 'https://github.com/networknuts/argocd.git' # Your Git repo URL
targetRevision: HEAD # Git branch, tag, or commit hash
path: ch-04 # Path within the repository containing the YAML files
destination:
server: 'https://kubernetes.default.svc' # Target cluster URL (this means the local cluster)
namespace: dev # Target namespace where resources will be deployed
syncPolicy:
automated: # Enables automated sync and self-healing
prune: true # Deletes resources from K8s if removed from Git
selfHeal: true # Overwrites manual changes made directly in the cluster
~
~


-rw-rw-r-- 1 alok alok 818 Aug 30 12:14 my-argo-app.yaml
alok@gateway:~/argocd$
alok@gateway:~/argocd$ kubectl apply -f my-argo-app.yaml
application.argoproj.io/webapp created
alok@gateway:~/argocd$

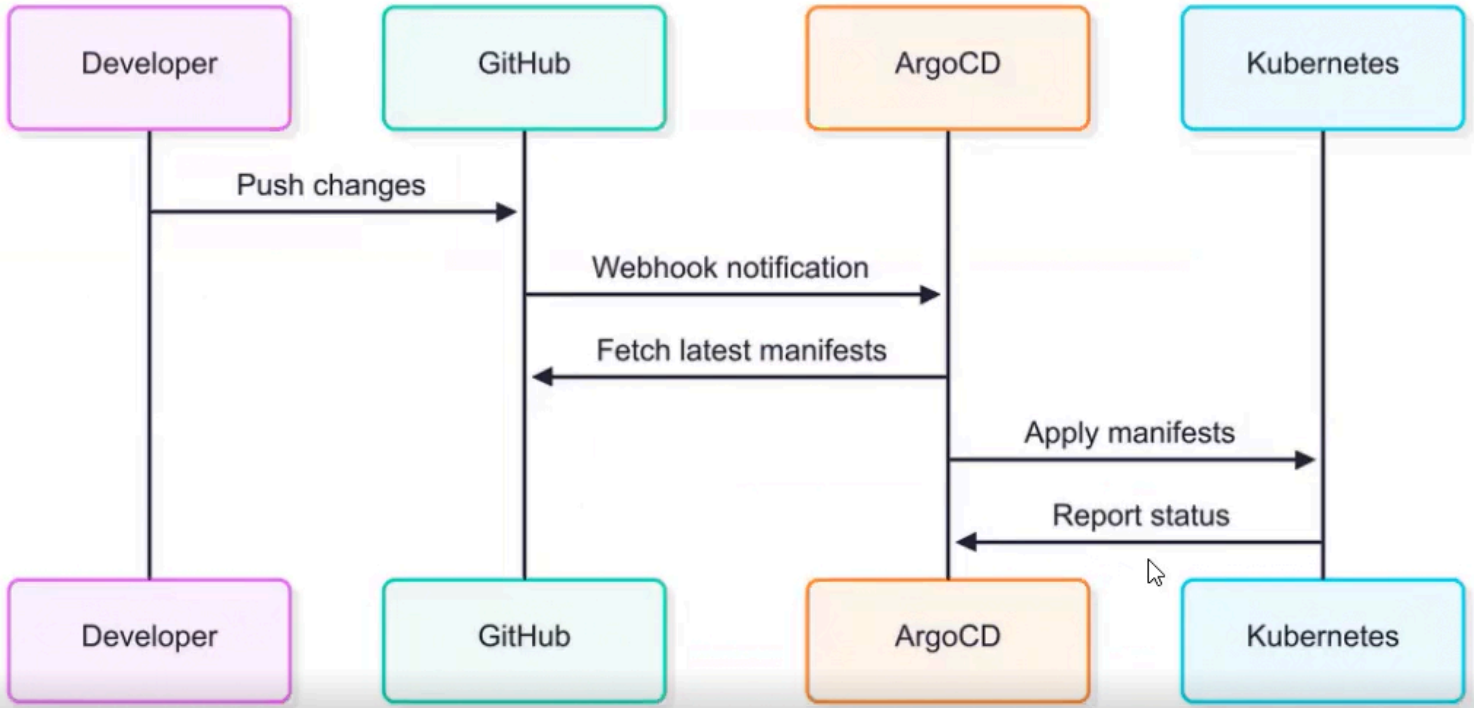

nginx-demo
Project: default
Labels:
Status: Healthy Synced
Repository: https://github.com/networknuts/arg...
Target Revision: HEAD
Path: ch-04
Destination: in-cluster
Namespace: default
Created At: 08/30/2026 17:38:06 (8 minutes ago)
Last Sync: 08/30/2026 17:40:50 (5 minutes ago)
SYNC
REFRESH
DELETE
webapp
Project: default
Labels:
Status: Healthy Synced
Repository: https://github.com/networknuts/arg...
Target Revision: HEAD
Path: ch-04
Destination: in-cluster
Namespace: dev
Created At: 08/30/2026 17:45:15 (a minute ago)
Last Sync: 08/30/2026 17:45:16 (a minute ago)
SYNC
REFRESH
DELETE


daemonsets ds apps/v1 true DaemonSet
deployments deploy apps/v1 true Deployment
replicasets rs apps/v1 true ReplicaSet
statefulsets sts apps/v1 true StatefulSet
applications app,apps argoproj.io/v1alpha1 true Application
applicationsets appset,appsets argoproj.io/v1alpha1 true ApplicationSe
appprojects appproj,appprojs argoproj.io/v1alpha1 true AppProject


ArgoCD
Sync Loop
network nuts
LOVE LEARN LINUX
Git State ≠ Cluster State
("OutOfSync")
Is Automated Sync Enabled?
No
Yes
Manual Approval Needed
click / cli
Apply Changes Automatically
Any Extra Options Set?
Prune: Enabled
Delete resources which are not in Git
SelfHeal: Enabled
Overwrite manual cluster changes
Retry: Enabled
Retries sync until it reaches the limit
#networknuts

GitOps in Action

Make some changes in our application and see how Argo CD will sync the application.



Troubleshooting Common Issues

Application Stuck in Progressing State. Run these commands to get more info about your app:
argocd app get argocd-demo
kubectl describe deployment argocd-demo
Look for typos in manifests or image pull issues.

Sync Failures
argocd app get nginx-demo
kubectl logs -n argocd deployment/argocd-application-controller

Out of Sync Status
Compare Git with the cluster state:
argocd app diff nginx-demo

Run a refresh:
argocd app get nginx-demo --refresh

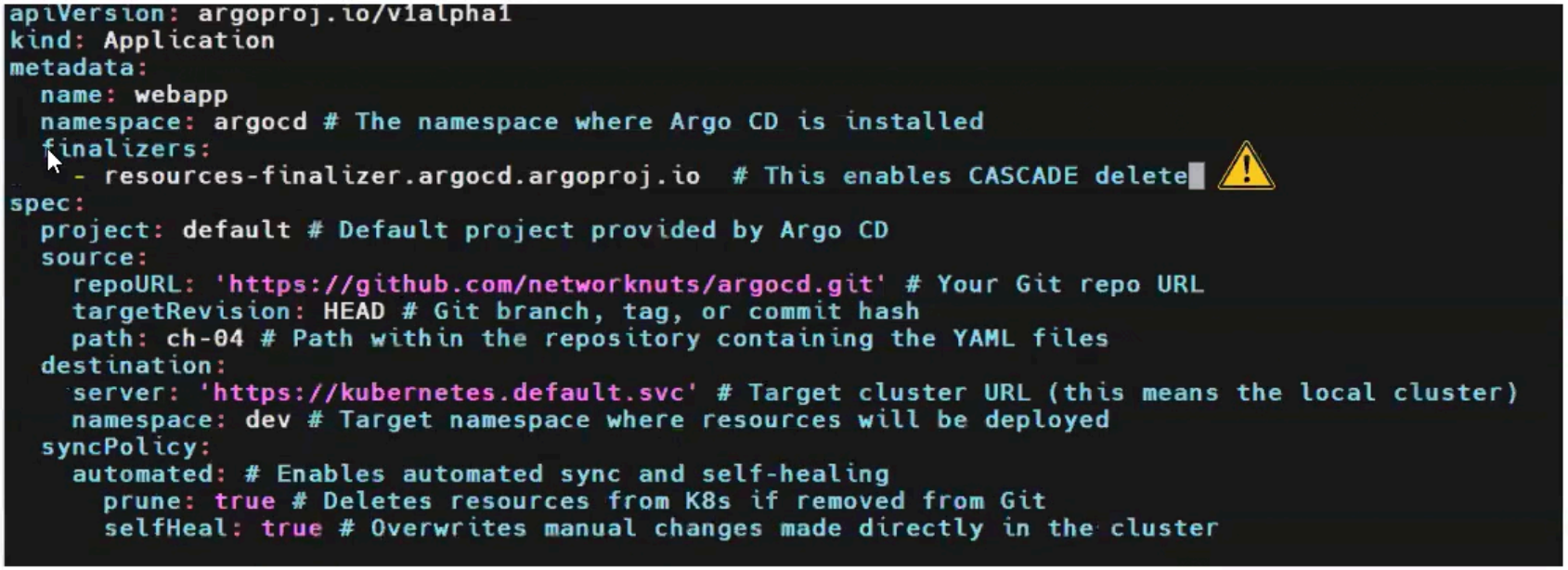
Deleting Application

Deleting an ArgoCD application sounds simple, but getting it wrong can either wipe out production workloads or leave behind orphaned resources cluttering your cluster.

When you delete an ArgoCD Application resource, two things can happen depending on your configuration:

1. **Cascade delete** - ArgoCD deletes the Application AND all Kubernetes resources it manages (Deployments, Services, ConfigMaps, etc.)
2. **Non-cascade delete** - ArgoCD deletes only the Application resource itself, leaving the managed Kubernetes resources running in the cluster

The behavior is controlled by finalizers on the Application resource and the deletion flags you pass.


apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
name: webapp
namespace: argocd # The namespace where Argo CD is installed
finalizers:
- resources-finalizer.argocd.argoproj.io # This enables CASCADE delete
spec:
project: default # Default project provided by Argo CD
source:
repoURL: 'https://github.com/networknuts/argocd.git' # Your Git repo URL
targetRevision: HEAD # Git branch, tag, or commit hash
path: ch-04 # Path within the repository containing the YAML files
destination:
server: 'https://kubernetes.default.svc' # Target cluster URL (this means the local cluster)
namespace: dev # Target namespace where resources will be deployed
syncPolicy:
automated: # Enables automated sync and self-healing
prune: true # Deletes resources from K8s if removed from Git
selfHeal: true # Overwrites manual changes made directly in the cluster

Deleting Application

Now Agro CD support two ways to delete applications:

Foreground
Background

The core difference between Argo CD foreground and background cascade deletion lies in **the order and execution speed of resource cleanup**.

- **Foreground deletion** ensures safety by completely deleting child resources before removing the parent Application object. (default)
- **Background deletion** prioritizes speed by immediately deleting the Application object and offloading child resource cleanup asynchronously.

Deleting Application

Foreground - Argo CD utilizes standard Kubernetes garbage collection logic. The root Application resource enters a Terminating state but will not disappear until every single resource it managed (Deployments, Pods, Services) has been successfully deleted from the cluster.

argocd app delete APPNAME --cascade --propagation-policy foreground

Background - the Application object is deleted instantly. The Kubernetes garbage controller then works behind the scenes to clean up the orphaned cluster resources asynchronously. While faster, there is a small risk that resource dependencies can stall in the background without clear visibility in the Argo CD dashboard.

argocd app delete APPNAME --cascade --propagation-policy background

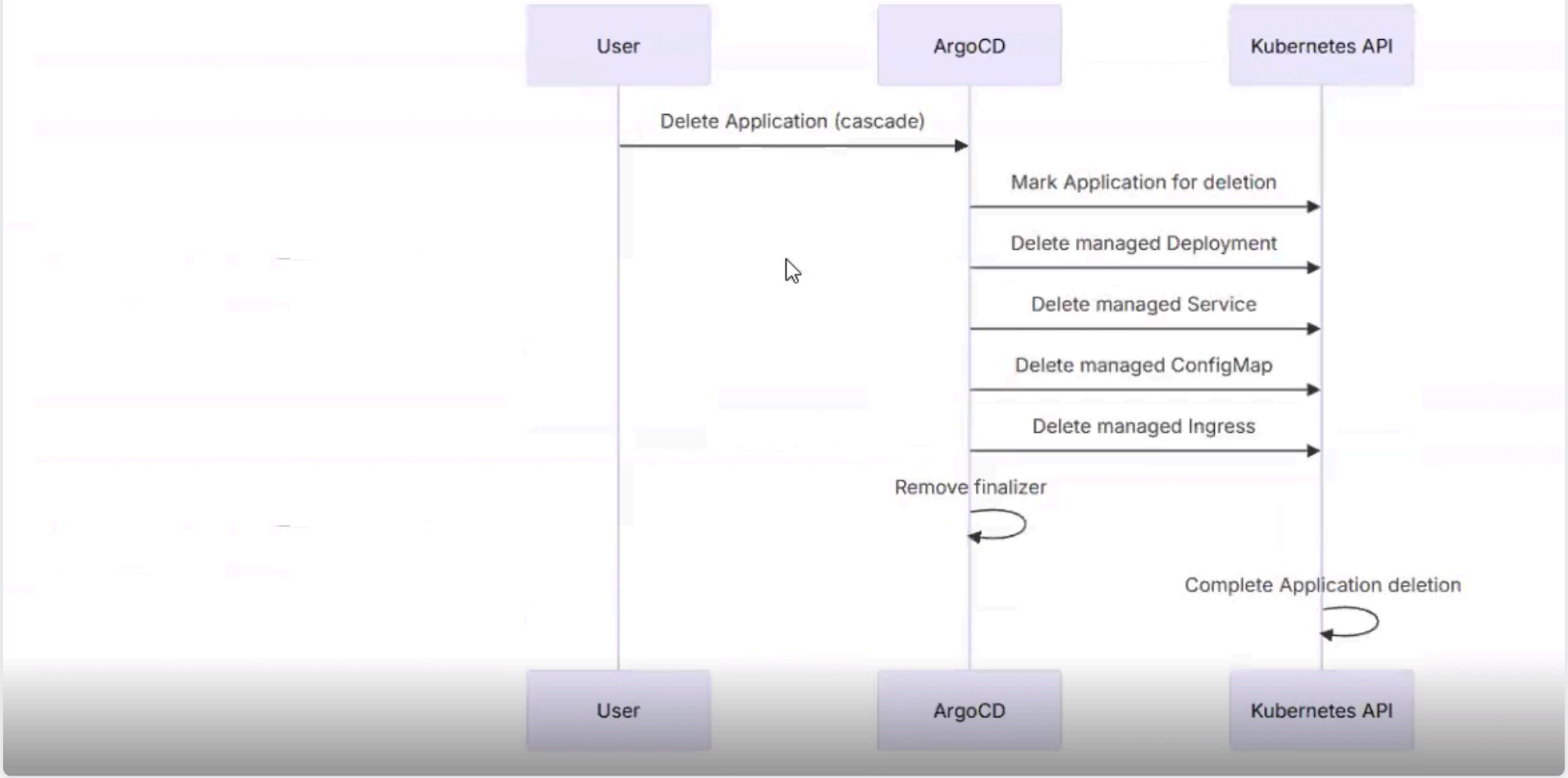
Deleting Application

How cascade delete works:

When you trigger a cascade delete, ArgoCD follows this sequence:

1. ArgoCD receives the delete request
2. It checks for the **resources-finalizer.argocd.argoproj.io** finalizer on the Application
3. If the finalizer is present, ArgoCD identifies all resources it tracks for that application
4. ArgoCD deletes each managed resource from the cluster
5. Once all resources are deleted (or deletion is initiated), ArgoCD removes the finalizer
6. Kubernetes then completes deletion of the Application resource itself

Deleting Application



Deleting Application from UI

One important caveat: cascade delete will remove PVCs if they are managed by ArgoCD. This means your persistent data is at risk:

Check if PVCs are tracked by ArgoCD before deleting
argocd app resources app-name | grep PersistentVolumeClaim